

Despliegue de una infraestructura segura en AWS con acceso remoto mediante VPN

Marta Bargues Bosch



ÍNDICE

1. Introducción	3
1.1 Contexto del proyecto	3
1.2 Descripción del proyecto	3
1.3 Objetivos del proyecto	4
1.4 Tecnologías utilizadas	5
1.5 Resultado esperado	6
2. Diseño de la arquitectura	7
2.1 Arquitectura general	7
2.2 Componentes de la infraestructura	7
2.3 Esquema de direccionamiento IP	8
2.4 Justificación del diseño	8
3. Preparación del entorno AWS	8
3.1 Creación de la cuenta AWS	8
3.2 Configuración inicial de la cuenta	10
3.4 Configuración de alertas de facturación y presupuesto	11
4. Creación de la red (VPC)	12
4.1 Creación de la VPC	12
4.2 Creación de la subred pública	14
4.3 Creación del Internet Gateway	15
4.4 Configuración de la tabla de enrutamiento	16
4.5 Configuración de la subred pública	17
5. Despliegue de la instancia EC2	18
5.1 Despliegue de la instancia EC2	18
6. Configuración inicial del servidor	20
6.1 Actualización del sistema	20
6.2 Hardening inicial del servidor	20
6.2.1 Configuración del firewall (UFW)	21
6.2.2 Instalación de Fail2ban	21
6.2.3 Configuración de Fail2ban	22
7. Implementación de la VPN WireGuard	23
7.1 Instalación de WireGuard	23
7.2 Generación de las claves criptográficas	24
7.3 Configuración del servidor WireGuard	25
7.4 Activación del reenvío IP (IP Forwarding)	26
7.6 Configuración de NAT para WireGuard	27
7.7 Configuración del cliente WireGuard	27
7.8 Conexión del cliente a la VPN	29
7.9. Incidencias encontradas y su resolución	31
7.9.1 Cambio de la dirección IP pública de la instancia	31

7.9.2 Puerto UDP 51820 bloqueado en el Security Group	31
7.9.3 Puerto bloqueado por UFW	32
7.9.4 Lecciones aprendidas	32
8. Despliegue de servicios y monitorización	33
8.1 Instalación de Docker	33
8.2 Despliegue de un servidor web con Nginx mediante Docker	34
8.2.1 Creación del contenedor	34
8.2.2 Publicación del servicio web	35
8.2.3 Personalización de la página web	36
8.3 Monitorización con Amazon CloudWatch	37
8.3.1 Creación del rol IAM	37
8.4.2 Asociación del rol a la instancia EC2	39
8.4.3 Instalación del agente de CloudWatch	39
8.4.4 Configuración del agente	40
8.4.5 Puesta en marcha del agente	42
8.4.6 Verificación de la monitorización	42
9. Conclusiones	45

1. Introducción

1.1 Contexto del proyecto

Aegis Systems es una empresa tecnológica ficticia dedicada a ofrecer soluciones informáticas a pequeñas y medianas empresas. Como parte de su crecimiento, ha decidido trasladar parte de su infraestructura a Amazon Web Services (AWS) para disponer de un entorno más seguro, flexible y fácil de administrar.

Para este proyecto se desplegará un servidor Ubuntu Server en Amazon EC2, aplicando diferentes medidas de seguridad para proteger la infraestructura. El acceso al servidor se realizará mediante una VPN WireGuard, utilizando además claves SSH, un firewall, medidas de hardening y la monitorización del sistema con Amazon CloudWatch.

1.2 Descripción del proyecto

Este proyecto tiene como objetivo diseñar e implementar una infraestructura segura en Amazon Web Services (AWS), aplicando conceptos de administración de sistemas, redes y ciberseguridad.

La infraestructura estará formada por una VPC, una máquina virtual Ubuntu Server en Amazon EC2 y una conexión remota segura mediante WireGuard VPN. Además, se aplicarán diferentes medidas de seguridad, como la autenticación multifactor (MFA), la creación de usuarios IAM con los permisos mínimos necesarios, el uso de claves SSH, la configuración del firewall y la monitorización de la infraestructura con Amazon CloudWatch.

Como parte del laboratorio, también se desplegarán varios servicios utilizando Docker, con el objetivo de simular un pequeño entorno empresarial. Por último, se realizarán diferentes pruebas para comprobar que todos los servicios funcionan correctamente y verificar que la infraestructura cumple las medidas de seguridad definidas al inicio del proyecto.

1.3 Objetivos del proyecto

Objetivo general

Diseñar e implementar una infraestructura segura en AWS que permita administrar un servidor Linux mediante una conexión VPN cifrada, aplicando buenas prácticas de administración de sistemas y ciberseguridad.

Objetivos específicos

- Crear una cuenta de AWS protegida con autenticación multifactor (MFA) y configurar usuarios IAM con los permisos necesarios.
- Diseñar la infraestructura de red utilizando una VPC y una subred pública.
- Desplegar una instancia Ubuntu Server en Amazon EC2.
- Configurar el acceso remoto mediante claves SSH.
- Implementar una VPN con WireGuard para administrar el servidor de forma segura.
- Aplicar medidas de seguridad y hardening sobre Ubuntu Server.
- Configurar un firewall para proteger el servidor frente a accesos no autorizados.
- Desplegar servicios utilizando Docker.
- Configurar la monitorización de la infraestructura con Amazon CloudWatch.
- Realizar pruebas para comprobar el funcionamiento de la infraestructura y verificar que las medidas de seguridad se han aplicado correctamente.

1.4 Tecnologías utilizadas

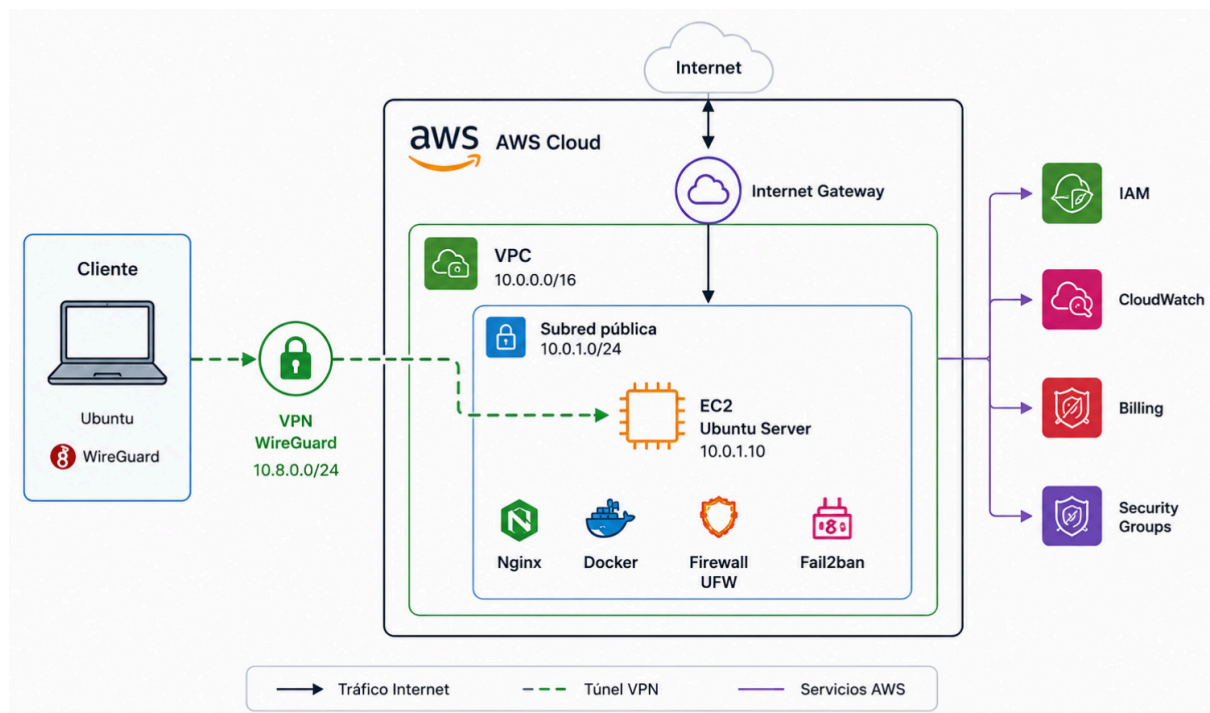
Tecnología	Función
AWS	Plataforma cloud
Amazon EC2	Servidor virtual
Amazon VPC	Red privada virtual
IAM	Gestión de identidades y permisos
Ubuntu Server 24.04 LTS	Sistema operativo del servidor
WireGuard	VPN para acceso seguro
SSH	Administración remota
Docker	Contenedores
Nginx	Servidor web de pruebas
UFW	Firewall del servidor
Fail2ban	Protección frente a ataques por fuerza bruta
Amazon CloudWatch	Monitorización y registros
Nmap	Auditoría y comprobación de la superficie de exposición

1.5 Resultado esperado

Al finalizar este proyecto se espera disponer de una infraestructura en AWS que sea funcional y, sobre todo, segura. El servidor podrá administrarse de forma remota a través de una VPN WireGuard, evitando que el acceso quede expuesto directamente a Internet.

A lo largo del proyecto se habrán implementado diferentes medidas de seguridad, como el acceso mediante claves SSH, la protección con firewall, la gestión de usuarios mediante IAM y la monitorización con Amazon CloudWatch, con el objetivo de mejorar la seguridad de la infraestructura.

Además, todo el proceso quedará documentado paso a paso para que pueda reproducirse fácilmente en el futuro y sirva como demostración de los conocimientos adquiridos durante el desarrollo del proyecto.



2. Diseño de la arquitectura

2.1 Arquitectura general

Antes de empezar a desplegar la infraestructura, es importante definir cómo estará organizada. Este diseño servirá como referencia durante todo el proyecto y ayudará a entender la función de cada uno de los componentes.

La infraestructura estará formada por una VPC con una única subred pública, donde se desplegará una instancia Ubuntu Server en Amazon EC2. El único servicio que estará accesible desde Internet será WireGuard, que permitirá establecer la conexión VPN con el servidor. Una vez conectado a la VPN, la administración se realizará mediante SSH, utilizando el túnel cifrado.

Además, en el servidor se desplegarán varios servicios con Docker y se aplicarán diferentes medidas de seguridad, como el uso de claves SSH, la configuración de un firewall, el hardening del sistema y la monitorización con Amazon CloudWatch.

2.2 Componentes de la infraestructura

Componente	Función
AWS	Plataforma cloud donde se desplegará la infraestructura
VPC	Red privada virtual que aislará la infraestructura
Internet Gateway	Permitirá la comunicación entre la VPC e Internet
Subred pública	Alojará la instancia EC2
EC2 Ubuntu Server	Servidor principal del laboratorio
Elastic IP	Dirección IP pública fija del servidor
Security Group	Firewall de red de la instancia EC2
WireGuard	VPN para la administración remota segura
Docker	Plataforma de contenedores para desplegar servicios
CloudWatch	Monitorización y recopilación de métricas y logs

2.3 Esquema de direccionamiento IP

Para hacer más fácil la administración y la identificación de los diferentes elementos de la infraestructura, se utilizarán las siguientes direcciones:

Elemento	Dirección
VPC	10.0.0.0/16
Subred pública	10.0.1.0/24
Servidor EC2 (IP privada)	10.0.1.10
Red VPN WireGuard	10.8.0.0/24

2.4 Justificación del diseño

La arquitectura se ha diseñado buscando un equilibrio entre simplicidad y seguridad. Se han utilizado solo los servicios necesarios para cumplir los objetivos del proyecto, procurando además que toda la infraestructura pueda desplegarse dentro del plan gratuito de AWS.

Para mantener el laboratorio lo más sencillo posible, se ha optado por una única subred pública. El acceso al servidor se realizará mediante una VPN WireGuard, evitando exponer los servicios de administración directamente a Internet y proporcionando una conexión segura.

Además, se utilizará Docker para desplegar los distintos servicios de forma sencilla y Amazon CloudWatch para supervisar el estado del servidor y disponer de información útil en caso de incidencias o problemas de funcionamiento.

3. Preparación del entorno AWS

3.1 Creación de la cuenta AWS

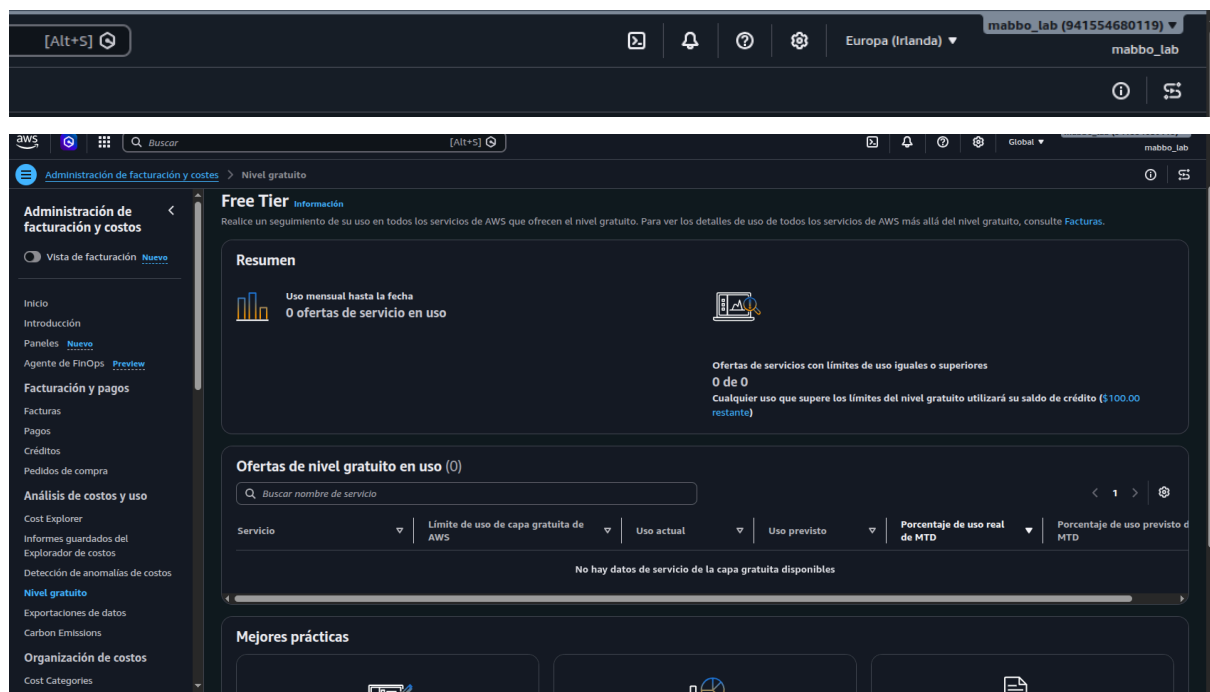
El primer paso del proyecto consistirá en crear una cuenta en Amazon Web Services (AWS), ya que será la plataforma sobre la que se desplegará toda la infraestructura. Para ello se utilizará una cuenta personal dentro del plan gratuito de AWS, suficiente para desarrollar el laboratorio sin generar costes, siempre que se respeten las limitaciones establecidas.

3.2 Configuración inicial de la cuenta

Una vez creada la cuenta de AWS, se realizará una configuración inicial para dejar el entorno preparado antes de empezar a desplegar la infraestructura.

Lo primero será proteger el usuario raíz activando la autenticación multifactor (MFA), ya que esta cuenta tiene acceso completo a todos los servicios de AWS y solo se utilizará cuando sea realmente necesario. Para el trabajo diario se creará más adelante un usuario IAM con los permisos adecuados.

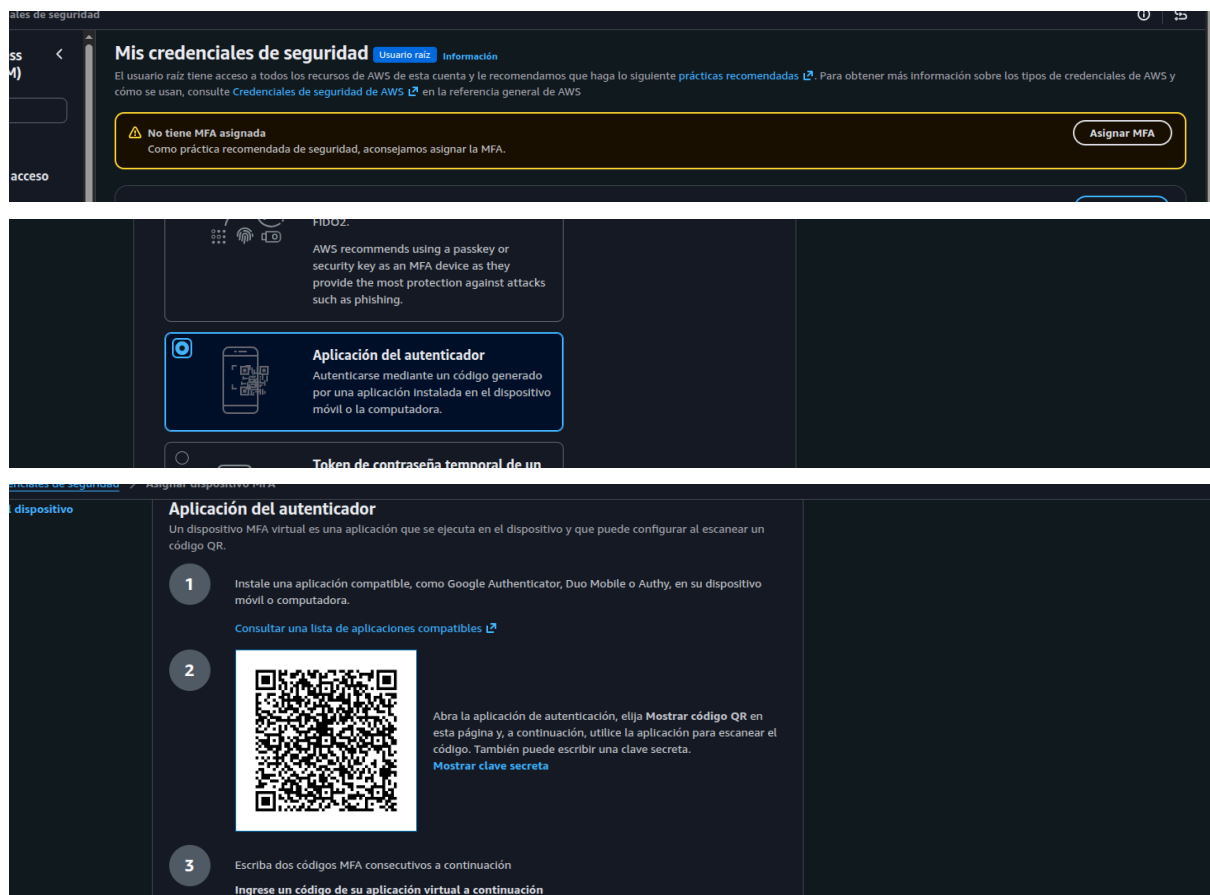
Después se configurará la región desde la que se trabajará, se ajustarán las opciones básicas de la consola y se revisará el panel de facturación para comprobar que el uso de los recursos se mantiene dentro de los límites del plan gratuito.



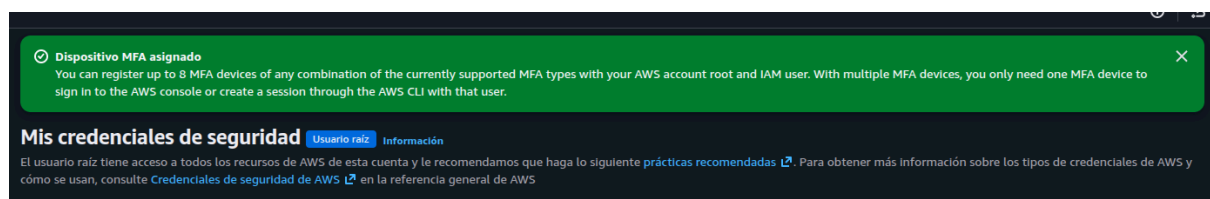
3.3 Activación de la autenticación multifactor (MFA)

Como primera medida de seguridad, se activará la autenticación multifactor (MFA) para el usuario de la cuenta de AWS. Este usuario dispone de acceso completo a todos los recursos, por lo que es recomendable añadir un segundo método de verificación además de la contraseña para evitar accesos no autorizados.

Para ello se accederá a las credenciales de seguridad del usuario y se asociará una aplicación de autenticación compatible, en nuestro caso Google Authenticator. Durante la configuración se escaneará el código QR proporcionado por AWS y se introducirán los códigos de verificación generados por la aplicación para completar el proceso.



Una vez finalizada la configuración, se comprobará que el dispositivo MFA ha quedado correctamente asociado y que su estado aparece como activo en la consola de AWS.

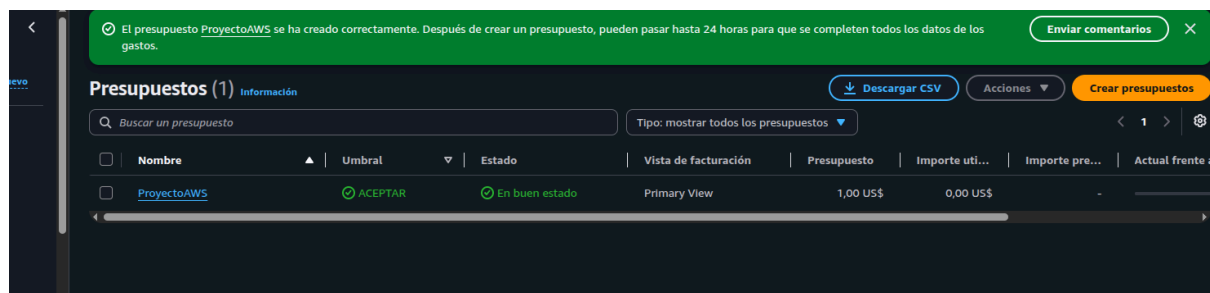
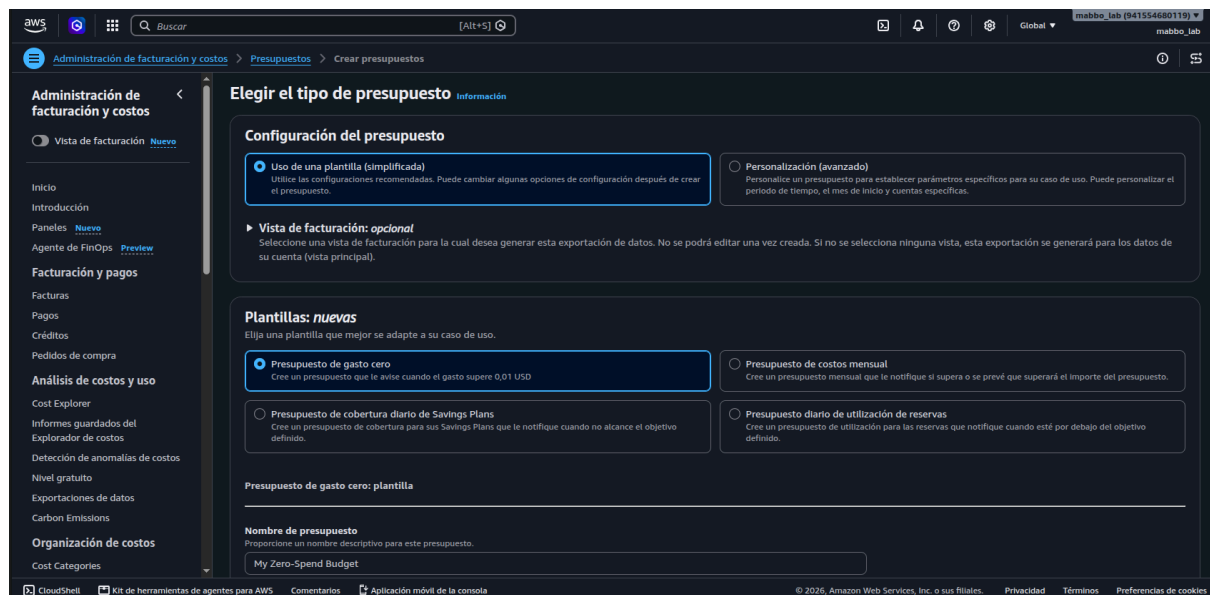
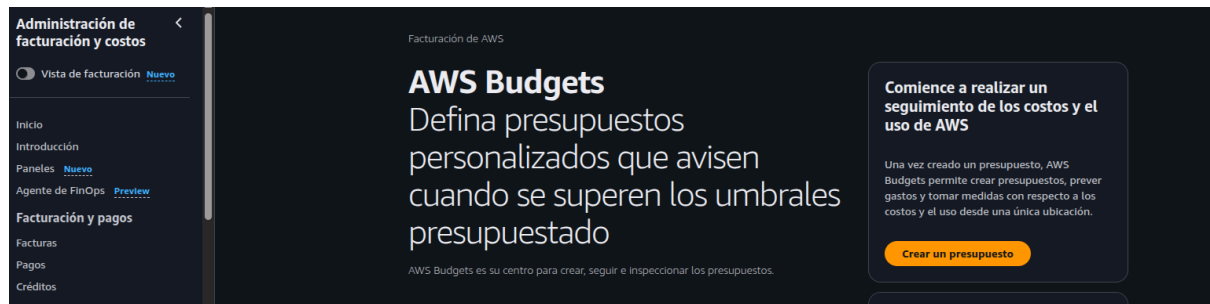


3.4 Configuración de alertas de facturación y presupuesto

Antes de comenzar a desplegar los recursos, se configurará un presupuesto en AWS para controlar el gasto de la cuenta y evitar costes inesperados durante el desarrollo del proyecto.

Aunque la infraestructura se diseñará para utilizar únicamente servicios incluidos en el nivel gratuito de AWS, se creará un presupuesto mensual mediante AWS Budgets. En este caso se establecerá un límite de 1 USD y se configurará una alerta que enviará una notificación por correo electrónico cuando el gasto alcance el 80 % del presupuesto.

De esta forma será posible controlar el consumo de la cuenta en todo momento y detectar rápidamente cualquier recurso que pueda generar costes imprevistos.



4. Creación de la red (VPC)

4.1 Creación de la VPC

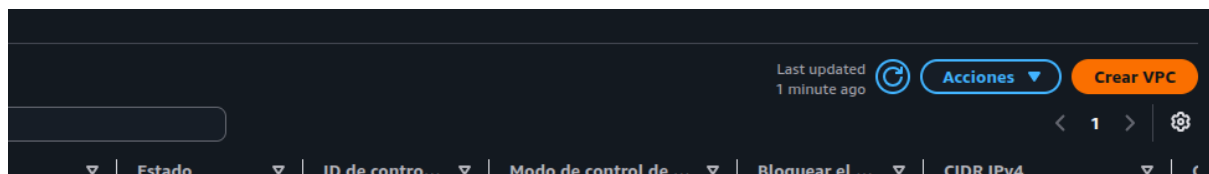
Una vez configurada la cuenta de AWS, se creará la red sobre la que se desplegará toda la infraestructura del proyecto.

Para ello se utilizará Amazon Virtual Private Cloud (VPC), un servicio que permite crear una red virtual aislada dentro de AWS. En esta red se desplegarán los distintos recursos del

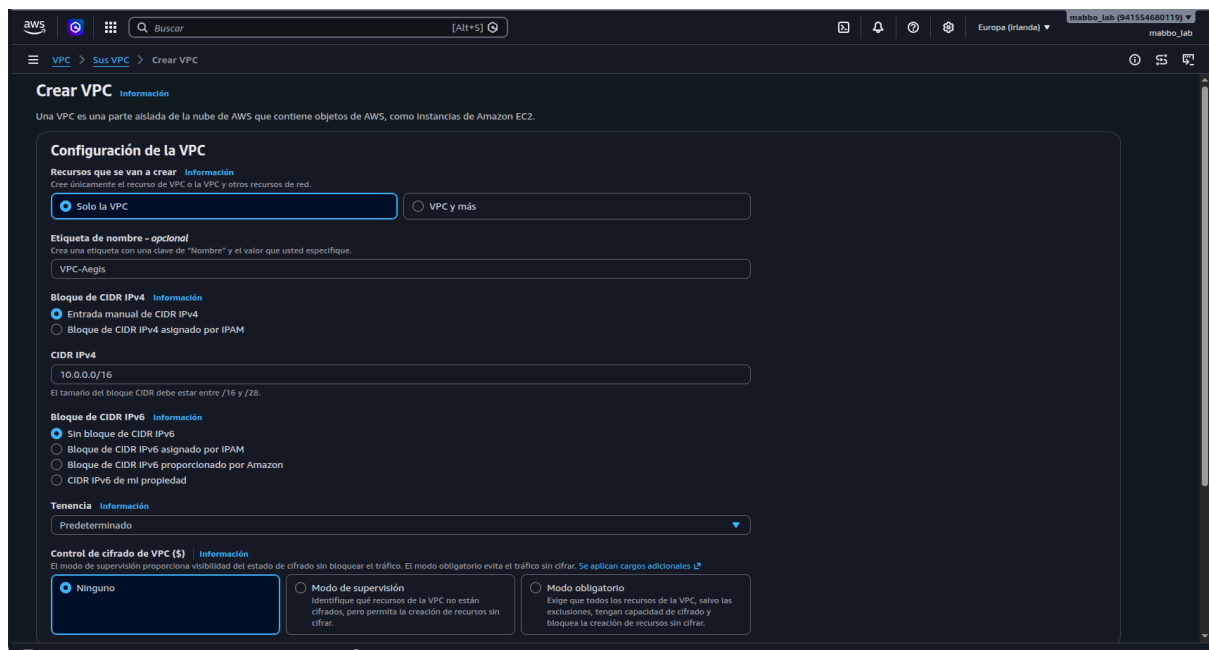
proyecto y se configurarán aspectos como el direccionamiento IP, las subredes y las reglas de comunicación entre los diferentes componentes.

En este caso se creará una única VPC con una subred pública, suficiente para alojar la instancia Ubuntu y permitir el acceso remoto mediante WireGuard. De esta forma se dispondrá de una arquitectura sencilla, fácil de administrar y compatible con los recursos incluidos en el nivel gratuito de AWS.

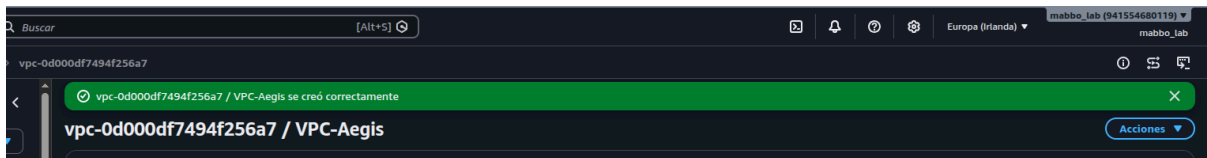
Para crear la red del proyecto, primero se accederá al servicio VPC desde el buscador de la consola de AWS. Una vez dentro, se entrará en el apartado Tus VPC y se seleccionará la opción Crear VPC.



Se asignará el nombre VPC-Aegis y se configurará el rango de direcciones 10.0.0.0/16, ya que ofrecerá suficiente espacio para crear nuevas subredes si fueran necesarias más adelante.



La VPC se creará únicamente con direcciones IPv4, sin configurar un bloque IPv6. También se mantendrá la tenencia predeterminada de AWS y no se habilitará el cifrado de la VPC, ya que para este proyecto no será necesario y podría generar costes. Por último, se revisará la configuración y se creará la VPC.

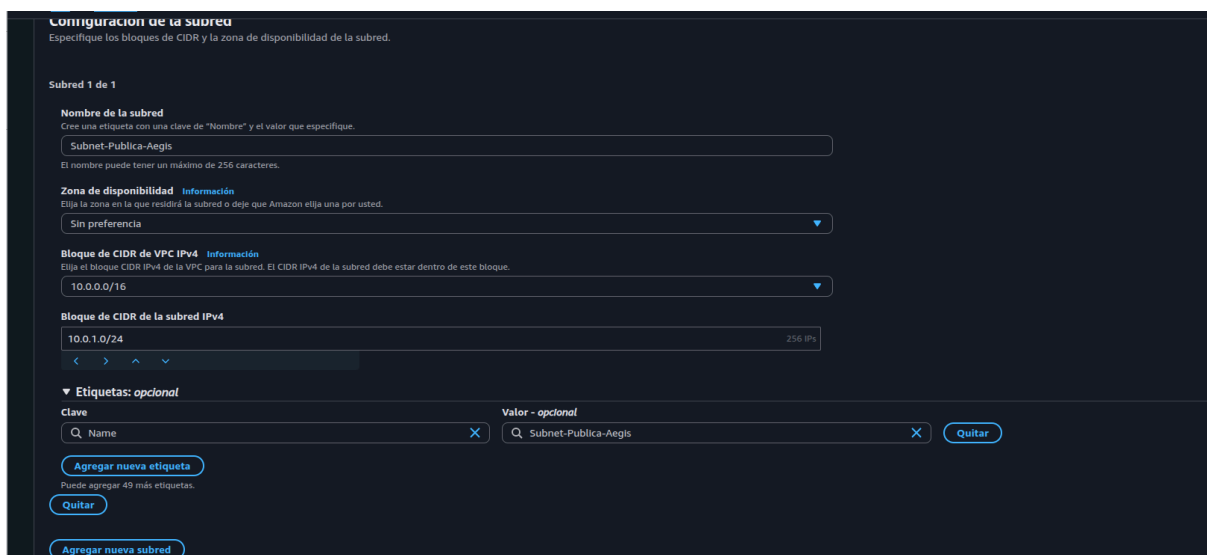


4.2 Creación de la subred pública

Una vez creada la VPC, se accederá al apartado Subredes del servicio VPC para crear la primera subred de la infraestructura.



Esta subred se asociará a VPC-Aegis y recibirá el nombre Subnet-Publica-Aegis. La zona de disponibilidad se dejará sin preferencia para que AWS seleccione automáticamente una dentro de la región elegida. Además, se configurará el bloque de direcciones 10.0.1.0/24, suficiente para el proyecto y permitir futuras ampliaciones.



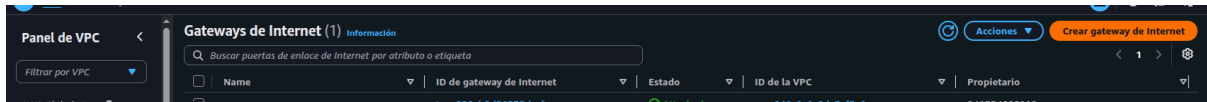
Al finalizar la configuración se creará la subred. Sin embargo, en este momento todavía no tendrá acceso a Internet. Para convertirla en una subred pública será necesario crear un Internet Gateway, asociarlo a la VPC, configurar una ruta hacia Internet y habilitar la asignación automática de direcciones IP públicas.



4.3 Creación del Internet Gateway

Una vez creada la subred, se configurará un Internet Gateway, que será el encargado de permitir la comunicación entre la VPC e Internet. Sin este componente, las instancias no podrán acceder a servicios externos ni recibir conexiones desde Internet, aunque dispongan de una dirección IP pública.

Para ello se accederá al apartado Internet Gateways del servicio VPC y se creará una nueva puerta de enlace con el nombre IGW-Aegis.



Crear gateway de Internet Información

Una gateway de Internet es un router virtual que conecta una VPC a Internet. Para crear una nueva gateway de Internet, especifique el nombre de la gateway a continuación.

Configuración de gateway de Internet

Etiqueta de nombre
Crea una etiqueta con una clave de "Nombre" y el valor que usted especifique.

Gateway-Aegis

Etiquetas: opcional
Una etiqueta es una marca que se asigna a un recurso de AWS. Cada etiqueta consta de una clave y un valor opcional. Puede utilizar las etiquetas para buscar y filtrar sus recursos o hacer un seguimiento de los costos de AWS.

Clave **Valor - opcional**

Q. Name X Q. Gateway-Aegis X Quitar

Agregar nueva etiqueta
Puede agregar 49 más etiquetas.

Cancelar **Crear gateway de Internet**

Una vez creada, se asociará a VPC-Aegis para que toda la infraestructura pueda utilizarla como salida hacia Internet.



Conectar a la VPC (igw-0544079acb4b90e30) Información

VPC
Conecte una gateway de Internet a la VPC para habilitar la comunicación con Internet. Especifique la VPC que desea asociar a continuación.

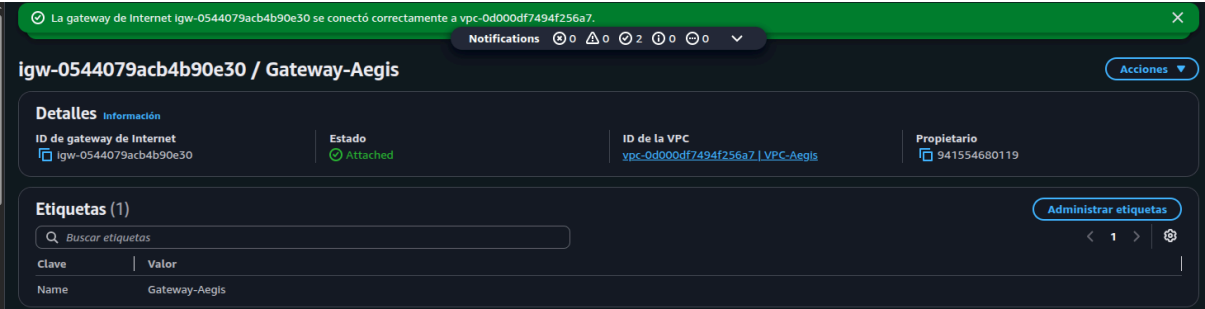
VPC disponibles
Conecte la gateway de Internet a esta VPC.

Q. vpc-0d000df7494f25ba7 X

► Comando de la interfaz de línea de comandos de AWS

Cancelar **Conectar gateway de Internet**

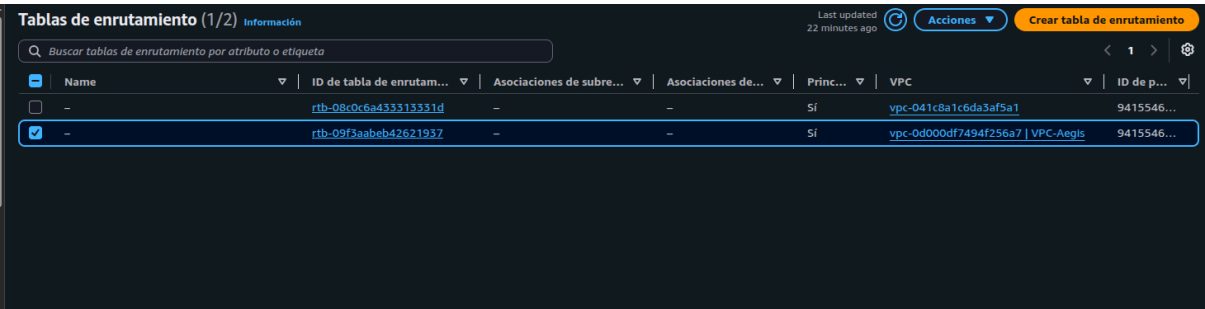
Al finalizar la configuración, se comprobará que el Internet Gateway aparece correctamente asociado a la VPC.



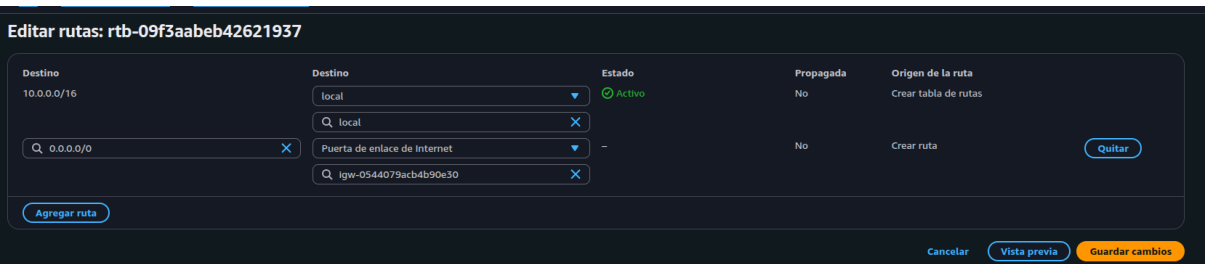
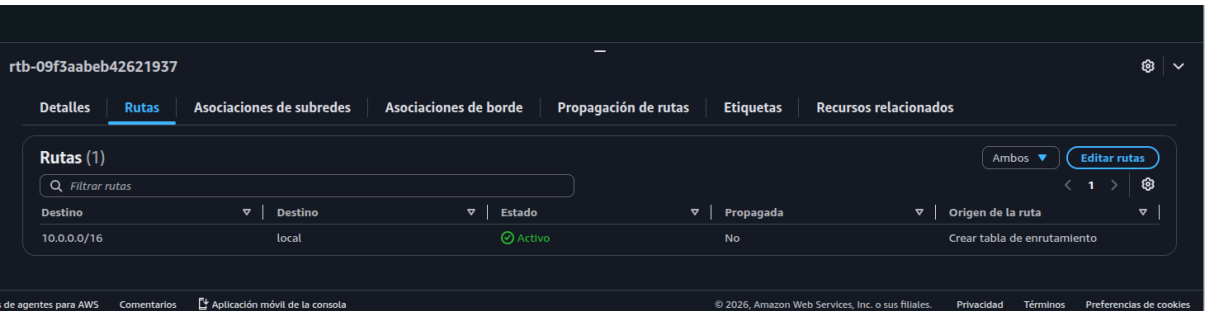
4.4 Configuración de la tabla de enrutamiento

Una vez asociado el Internet Gateway a la VPC, será necesario configurar la tabla de enrutamiento para que la subred pueda comunicarse con Internet.

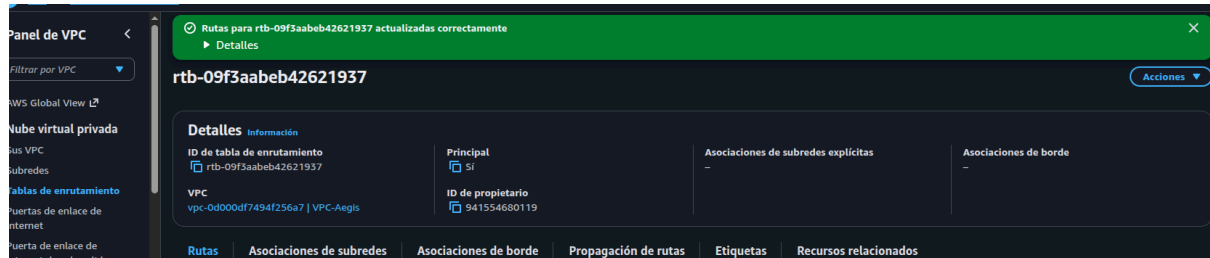
Para ello se accederá al apartado Tablas de enrutamiento del servicio VPC y se seleccionará la tabla correspondiente a VPC-Aegis.



A continuación, se añadirá una nueva ruta con destino 0.0.0.0/0, utilizando IGW-Aegis como puerta de enlace. Esta ruta permitirá que el tráfico dirigido a Internet salga a través del Internet Gateway.



Después, la misma tabla de enrutamiento se asociará a Subnet-Publica-Aegis, de forma que la subred pueda utilizar las rutas configuradas. La ruta local creada automáticamente por AWS se mantendrá sin modificar, ya que permite la comunicación entre los recursos de la propia VPC.



Al finalizar la configuración, la tabla de enrutamiento contará con la ruta local de la VPC y con una segunda ruta hacia Internet a través de IGW-Aegis, dejando la subred preparada para disponer de conectividad externa.

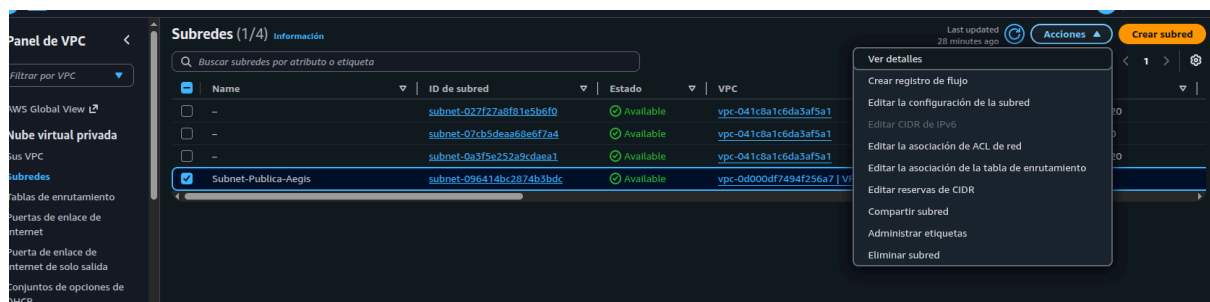
4.5 Configuración de la subred pública

Para terminar la configuración de la red, se habilitará la asignación automática de direcciones IPv4 públicas en Subnet-Publica-Aegis.

Con este ajuste, cualquier instancia EC2 que se cree dentro de esta subred recibirá automáticamente una dirección IP pública, por lo que será posible conectarse a ella desde Internet sin tener que realizar ninguna configuración adicional.

Este paso facilitará la primera conexión al servidor mediante SSH. Más adelante, cuando WireGuard esté configurado, el acceso al servidor se realizará a través de la VPN, sin depender de la dirección IP pública.

Para realizar esta configuración, se accederá al servicio VPC y, dentro del apartado Subredes, se seleccionará Subnet-Publica-Aegis. A continuación, desde el menú Acciones, se abrirá la opción Editar configuración de la subred.



Una vez dentro, se habilitará la opción Asignación automática de dirección IPv4 pública y se guardarán los cambios para aplicar la configuración.

Editar la configuración de la subred [Información](#)

Subred
ID de subred
subnet-096414bc2874b3bdc

Nombre
Subnet-Publica-Aegis

Configuración de la asignación automática de IP [Información](#)
Permita que AWS asigne automáticamente una dirección IPv4 o IPv6 pública a una nueva interfaz de red principal para una instancia de esta subred.

☒ **Habilitar la asignación automática de la dirección IPv4 pública** [Información](#)
☐ **Habilitar la asignación automática de direcciones IPv4 propiedad del cliente** [Información](#)
Opción desactivada porque no se encontraron grupos propiedad del cliente.

Configuración del nombre basado en recursos (RBN) [Información](#)

Panel de VPC [Información](#)

Subredes (1/4)

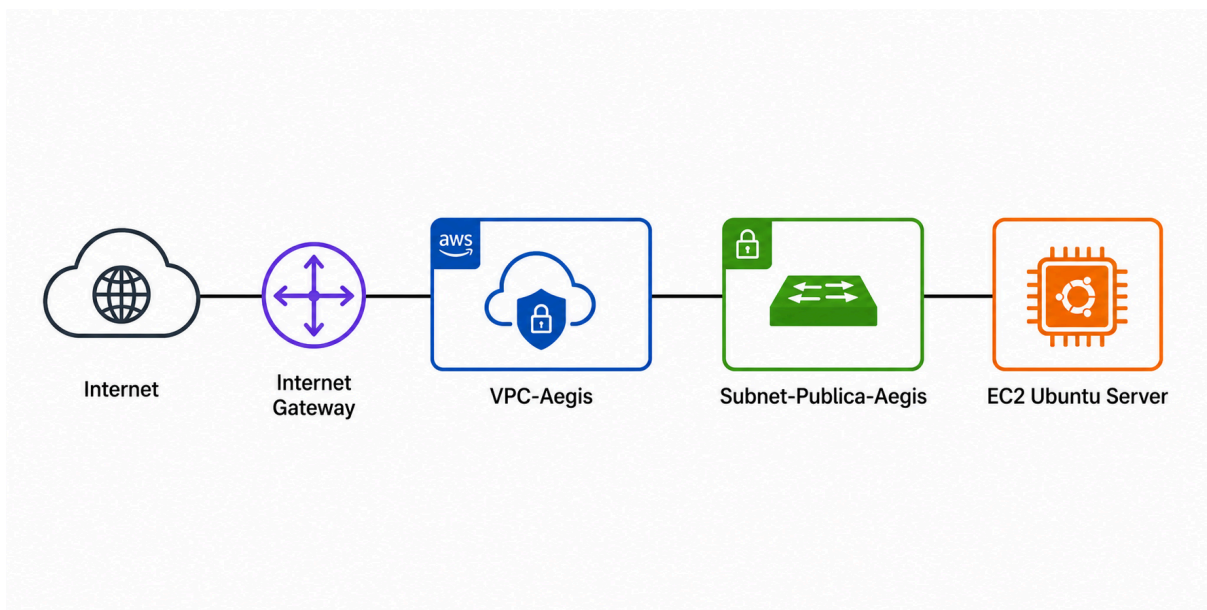
Ha cambiado correctamente la configuración de la subred:
Habilitar la asignación automática de la dirección IPv4 pública

Buscar subredes por atributo o etiqueta

	Name	ID de subred	Estado	VPC	Bloquear el ...	CIDR IPv4
☐	-	subnet-027f27a8fa1e5b6f0	Available	vpc-041c8a1c6da3af5a1	Desactivado	172.31.32.0/20
☐	-	subnet-07c5deaa68e6f7a4	Available	vpc-041c8a1c6da3af5a1	Desactivado	172.31.0.0/20
☐	-	subnet-0a3f5e252a9cdadaa1	Available	vpc-041c8a1c6da3af5a1	Desactivado	172.31.16.0/20
☒	Subnet-Publica-Aegis	subnet-096414bc2874b3bdc	Available	vpc-0d000df7494f25ba7 VPC-Aegis	Desactivado	10.0.1.0/24

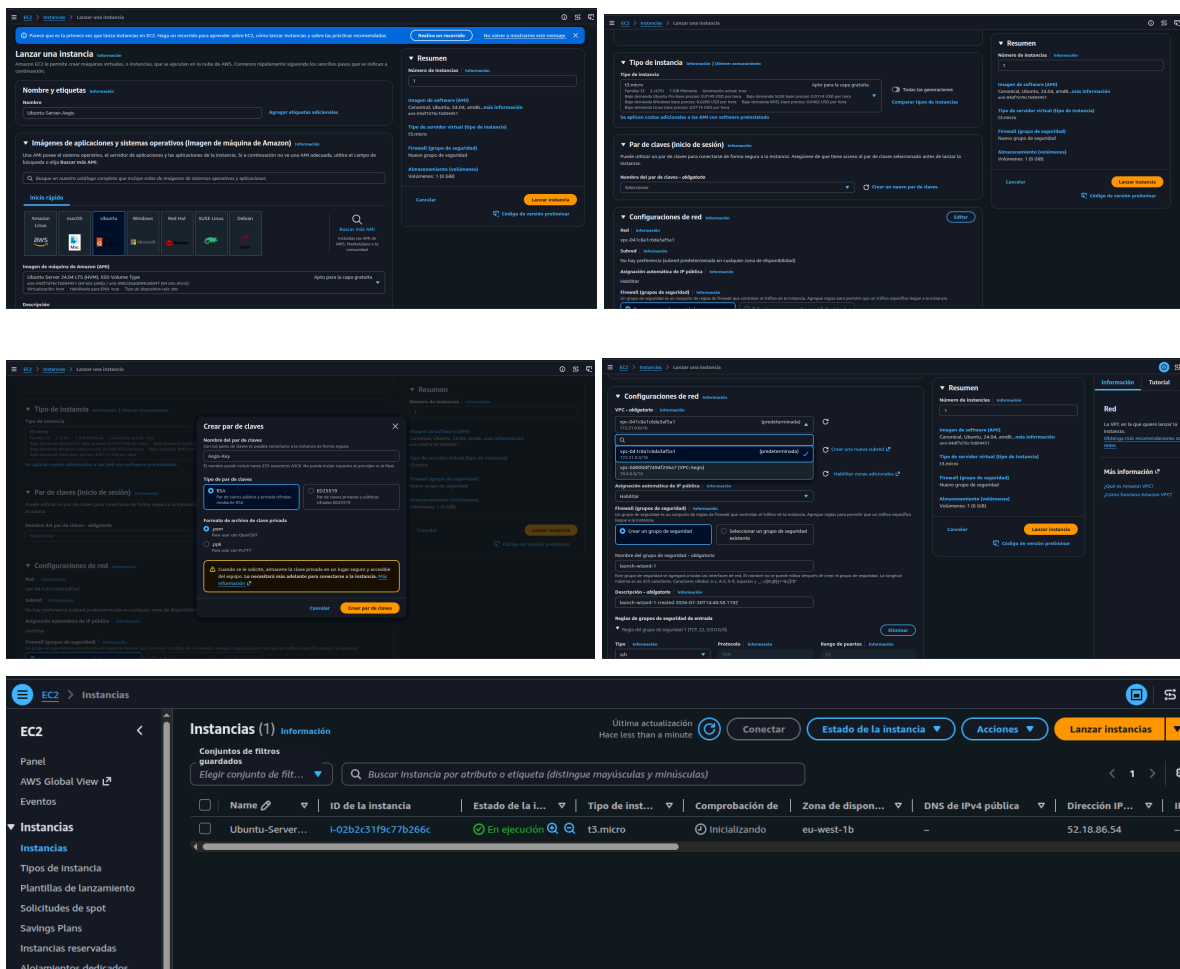
5. Despliegue de la instancia EC2

5.1 Despliegue de la instancia EC2



Una vez configurada la red, se desplegará una instancia Amazon EC2 que actuará como servidor principal del proyecto.

Para ello se utilizará Ubuntu Server, una distribución Linux muy utilizada tanto en entornos profesionales como en la nube. Durante el despliegue se configurarán la red, la subred, el grupo de seguridad, el almacenamiento y el par de claves SSH, que permitirá acceder al servidor de forma segura.



Sobre esta instancia se instalarán más adelante WireGuard, Docker, Nginx y el resto de servicios necesarios para el proyecto. Al principio, el servidor se administrará mediante SSH. Más adelante, cuando WireGuard esté configurado, el acceso se realizará a través de la VPN, evitando que el servidor tenga que estar expuesto directamente a Internet.

Para conectarnos usaremos la clave SSH que hemos creado y la IP pública de la máquina.

```
ubuntu@ip-10-0-1-160:~$ ssh -i /home/ubuntu/.ssh/Ubuntu-Server-01.pem ubuntu@52.18.86.54
The authenticity of host '52.18.86.54 (52.18.86.54)' can't be established.
ED25519 key fingerprint is SHA256:5/HHj3yZMcNLL24LE99mcx02LcVNS807/8cK114N8.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '52.18.86.54' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.17.0-1017-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Thu Jul 30 14:53:02 UTC 2026

System load:  0.14           Temperature:   -273.1 C
Usage of /:   26.7% of 6.71GB Processes:      113
Memory usage: 25%          Users logged in: 0
Swap usage:   0%           IPv4 address for ens5: 10.0.1.160

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/*copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-10-0-1-160:~$
```

6. Configuración inicial del servidor

6.1 Actualización del sistema

Antes de empezar a instalar los distintos servicios, se actualizará el servidor para asegurarse de que todos los paquetes se encuentran en su versión más reciente.

Una vez establecida la conexión mediante SSH, se ejecutarán los siguientes comandos para actualizar el sistema, instalar las últimas correcciones disponibles y eliminar los paquetes que ya no sean necesarios:

```
sudo apt update
sudo apt full-upgrade -y
sudo reboot
```

Cuando el servidor se reinicie, se volverá a acceder por SSH y se comprobarán la versión de Ubuntu y la del kernel para documentar.

```
lsb_release -a
uname -r
```



```
ubuntu@ip-10-0-1-160:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.4 LTS
Release:        24.04
Codename:       noble
ubuntu@ip-10-0-1-160:~$ uname -r
7.0.0-1009-aws
ubuntu@ip-10-0-1-160:~$
```

6.2 Hardening inicial del servidor

Una vez actualizado el sistema, se aplicarán las primeras medidas de seguridad para dejar el servidor preparado antes de instalar el resto de servicios del proyecto.

En este apartado se configurarán diferentes opciones para mejorar la seguridad del servidor, como el firewall, el acceso mediante SSH y la protección frente a intentos de acceso no autorizados. Con estos cambios se reducirá la exposición del servidor a posibles ataques desde Internet y se dispondrá de una base más segura sobre la que continuar con la instalación de WireGuard, Docker y el resto de herramientas.

6.2.1 Configuración del firewall (UFW)

Ubuntu incorpora Uncomplicated Firewall (UFW), una herramienta que simplifica la administración del firewall del sistema.

Se comprobará si está instalado y se configuraran las reglas como; bloquear conexiones entrantes por defecto, permitir conexiones salientes o permitir SSH.

```
ubuntu@ip-10-0-1-160:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
ubuntu@ip-10-0-1-160:~$ sudo ufw allow OpenSSH
Rules updated
Rules updated (v6)
ubuntu@ip-10-0-1-160:~$ sudo ufw default deny incoming
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
ubuntu@ip-10-0-1-160:~$
```

Se activará el firewall y se comprobará la configuración.

```
ubuntu@ip-10-0-1-160:~$ sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
ubuntu@ip-10-0-1-160:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp (OpenSSH) ALLOW IN Anywhere
22/tcp (OpenSSH (v6)) ALLOW IN Anywhere (v6)
```

6.2.2 Instalación de Fail2ban

Para reforzar la seguridad del servidor, se instalará Fail2ban, una herramienta que ayuda a proteger el acceso mediante SSH frente a intentos de fuerza bruta.

Su funcionamiento consiste en supervisar los registros del sistema y detectar intentos repetidos de inicio de sesión. Si una misma dirección IP supera el número de intentos permitidos, Fail2ban la bloqueará automáticamente durante un tiempo determinado, reduciendo el riesgo de ataques automatizados.

Para instalar Fail2ban, primero se ejecutará el siguiente comando:

`sudo apt install fail2ban -y`

```
ubuntu@ip-10-0-1-160:~$ sudo apt install fail2ban -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  python3-pyasyncore python3-pyinotify whois
Suggested packages:
  mailx monit sqlite3 python-pyinotify-doc
The following NEW packages will be installed:
  fail2ban python3-pyasyncore python3-pyinotify whois
0 upgraded, 4 newly installed, 0 to remove and 0 not upgraded.
Need to get 496 kB of archives.
After this operation, 2572 kB of additional disk space will be used.
Get:1 http://security.ubuntu.com/ubuntu-security/ubuntu-security/ubuntu-security-1.0.2.2 [10.1 kB]
```

Una vez finalizada la instalación, se comprobará que el servicio se ha iniciado correctamente con:

`sudo systemctl status fail2ban`

```
ubuntu@ip-10-0-1-160:~$ sudo systemctl status fail2ban
● fail2ban.service - Fail2Ban Service
   Loaded: loaded (/usr/lib/systemd/system/fail2ban.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-07-31 12:10:01 UTC; 40s ago
     Docs: man:fail2ban(1)
    Main PID: 1548 (fail2ban-server)
      Tasks: 5 (limit: 627)
    Memory: 23.0M (peak: 23.3M)
       CPU: 1.039s
    CGroup: /system.slice/fail2ban.service
            └─1548 /usr/bin/python3 /usr/bin/fail2ban-server -xf start

Jul 31 12:10:01 ip-10-0-1-160 systemd[1]: Started fail2ban.service - Fail2Ban Service.
Jul 31 12:10:01 ip-10-0-1-160 fail2ban-server[1548]: 2026-07-31 12:10:01.892 fail2ban.configreader [1548]: WARNING 'allowip_v6' not defined in 'Def
```

Después se habilitará su inicio automático para que se ejecute cada vez que el servidor se reinicie:

`sudo systemctl enable fail2ban`

```
ubuntu@ip-10-0-1-160:~$ sudo systemctl enable fail2ban
Synchronizing state of fail2ban.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable fail2ban
ubuntu@ip-10-0-1-160:~$
```

Por último, se verificará que Fail2ban está funcionando correctamente y que la protección para SSH se encuentra activa:

`sudo fail2ban-client status`

```
ubuntu@ip-10-0-1-160:~$ sudo fail2ban-client status
Status
|- Number of jail:      1
|- Jail list:          sshd
ubuntu@ip-10-0-1-160:~$
```

6.2.3 Configuración de Fail2ban

Una vez instalado Fail2ban, se configurará para proteger el servicio SSH frente a intentos repetidos de inicio de sesión.

Para ello se creará el archivo `jail.local`, donde se almacenará la configuración personalizada. Se utilizará este archivo en lugar de modificar `jail.conf`, ya que así la configuración se conservará aunque Fail2ban se actualice.

El archivo se creará con el siguiente comando:

`sudo nano /etc/fail2ban/jail.local`

```
GNU nano 7.2 /etc/fail2ban/jail.local *
[DEFAULT]
bantime = 1h
findtime = 10m
maxretry = 5

[sshd]

enabled = true
port = ssh
logpath = %(sshd_log)s
backend = systemd
```

Con esta configuración, una dirección IP quedará bloqueada durante una hora si realiza cinco intentos fallidos de inicio de sesión en un intervalo de diez minutos.

Una vez guardado el archivo, se reiniciará el servicio para aplicar los cambios:

```
sudo systemctl restart fail2ban
```

Por último, se comprobará que la configuración se ha aplicado correctamente con los siguientes comandos:

```
sudo fail2ban-client status
```

```
sudo fail2ban-client status sshd
```

```
ubuntu@ip-10-0-1-160:~$ sudo fail2ban-client status
Status
|- Number of jail:      1
|- Jail list:  sshd
ubuntu@ip-10-0-1-160:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 0
| |- Total failed:    0
| '- Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd
'- Actions
  |- Currently banned: 0
  |- Total banned:    0
  '- Banned IP list:
ubuntu@ip-10-0-1-160:~$
```

El primer comando mostrará que la protección para SSH está activa, mientras que el segundo permitirá comprobar el estado del jail y el número de intentos fallidos o direcciones IP bloqueadas.

7. Implementación de la VPN WireGuard

7.1 Instalación de WireGuard

Una vez configurado el servidor, se instalará WireGuard, que será la herramienta utilizada para acceder al servidor de forma segura desde el equipo cliente.

WireGuard es una VPN ligera, rápida y fácil de configurar, por lo que resulta una buena opción para este proyecto. Gracias a ella se establecerá una conexión cifrada entre el equipo cliente y el servidor Ubuntu desplegado en AWS.

Al principio, el acceso al servidor se realizará mediante SSH para completar la configuración. Cuando WireGuard esté funcionando correctamente, el acceso pasará a realizarse a través de la VPN, evitando dejar el servicio SSH accesible directamente desde Internet.

Para instalar WireGuard, se ejecutará el siguiente comando:

`sudo apt install wireguard -y`

```
ubuntu@ip-10-0-1-160:~$ sudo apt install wireguard -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  wireguard-tools
The following NEW packages will be installed:
  wireguard wireguard-tools
0 upgraded, 2 newly installed, 0 to remove and 0 not upgraded.
Need to get 92.2 kB of archives.
```

Cuando finalice la instalación, se comprobará que se ha instalado correctamente con:

`wg --version`

```
ubuntu@ip-10-0-1-160:~$ wg --version
wireguard-tools v1.0.20210914 - https://git.zx2c4.com/wireguard-tools/
ubuntu@ip-10-0-1-160:~$
```

Después, se cargará el módulo del kernel con el siguiente comando:

`sudo modprobe wireguard`

Por último, se comprobará el estado del servicio:

`systemctl status wg-quick@wg0`

```
ubuntu@ip-10-0-1-160:~$ sudo modprobe wireguard
ubuntu@ip-10-0-1-160:~$ systemctl status wg-quick@wg0
○ wg-quick@wg0.service - WireGuard via wg-quick(8) for wg0
   Loaded: loaded (/usr/lib/systemd/system/wg-quick0.service; disabled; preset: enabled)
   Active: inactive (dead)
     Docs: man:wg-quick(8)
           man:wg(8)
           https://www.wireguard.com/
           https://www.wireguard.com/quickstart/
           https://git.zx2c4.com/wireguard-tools/about/src/man/wg-quick.8
           https://git.zx2c4.com/wireguard-tools/about/src/man/wg.8
ubuntu@ip-10-0-1-160:~$
```

En este momento es normal que aparezca un mensaje indicando que el servicio no existe. Esto se debe a que todavía no se ha creado el archivo de configuración wg0.conf.

7.2 Generación de las claves criptográficas

Se generará el par de claves pública y privada que utilizará el servidor para autenticarse dentro de la VPN.

Se crea el directorio si no existe y se cambian los permisos.

```
ubuntu@ip-10-0-1-160:~$ sudo mkdir /etc/wireguard
mkdir: cannot create directory '/etc/wireguard': File exists
ubuntu@ip-10-0-1-160:~$ sudo chmod 700 /etc/wireguard/
ubuntu@ip-10-0-1-160:~$
```

Se generará la clave privada , pública y se protegerán.

```
ubuntu@ip-10-0-1-160:~$ wg genkey | sudo tee /etc/wireguard/server_private.key
-----
ubuntu@ip-10-0-1-160:~$ sudo cat /etc/wireguard/server_private.key | wg pubkey | sudo tee /etc/wireguard/server_public.key
dHChEGUjRwHRZL8VYQCc3j050KhRaE8LYzJeXga8nzc=
-----
ubuntu@ip-10-0-1-160:~$ sudo chmod 600 /etc/wireguard/server_private.key
ubuntu@ip-10-0-1-160:~$ sudo chmod 600 /etc/wireguard/server_public.key
```

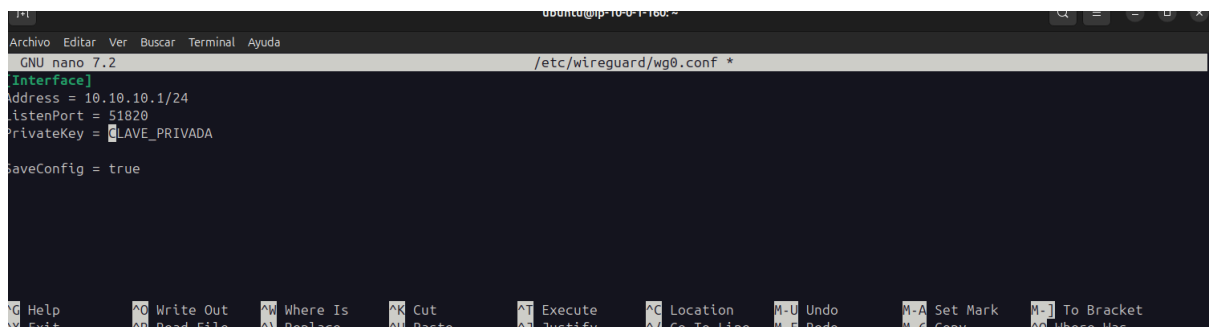
7.3 Configuración del servidor WireGuard

Una vez instalado WireGuard, se configurará el servidor creando el archivo wg0.conf, donde se definirán los parámetros necesarios para poner en marcha la VPN.

En este archivo se configurarán la clave privada del servidor, la dirección IP que utilizará dentro de la VPN y el puerto por el que aceptará las conexiones de los clientes autorizados.

Para la red de la VPN se utilizará el rango 10.10.10.0/24, independiente de la red de la VPC de AWS (10.0.0.0/16). De esta forma se evitarán posibles conflictos entre ambas redes y la comunicación será más sencilla de gestionar.

Para crear el archivo wg0.conf usaremos el comando:
sudo nano /etc/wireguard/wg0.conf



```
GNU nano 7.2 /etc/wireguard/wg0.conf *
[Interface]
address = 10.10.10.1/24
listenPort = 51820
PrivateKey = CLAVE_PRIVADA
SaveConfig = true
```

En la parte donde aparece “CLAVE_PRIVADA” pegaremos nuestra clave generada guardamos y salimos.

Después protegeremos el archivo de configuración con permisos y terminamos con esta parte, para comprobar que todo funciona correctamente arrancaremos WireGuard.

```
ubuntu@ip-10-0-1-160:~$ sudo systemctl start wg-quick@wg0
ubuntu@ip-10-0-1-160:~$ sudo wg
Interface: wg0
  public key: 73lcEe4aKSnpG93BzfJ/sLMampFQy84hF9yT3Shguj4=
  private key: (hidden)
  listening port: 51820
ubuntu@ip-10-0-1-160:~$
```

7.4 Activación del reenvío IP (IP Forwarding)

Para que la VPN funcione correctamente, será necesario habilitar el reenvío de paquetes en el servidor. Sin esta configuración, los clientes podrán conectarse a WireGuard, pero no podrán acceder a la red de la VPC ni comunicarse con otros equipos.

Lo primero será comprobar el estado actual del reenvío de paquetes con el siguiente comando:

```
cat /proc/sys/net/ipv4/ip_forward
```

```
ubuntu@ip-10-0-1-160:~$ cat /proc/sys/net/ipv4/ip_forward
0
ubuntu@ip-10-0-1-160:~$
```

El resultado es 0, así que significa que el reenvío está desactivado. Para activarlo temporalmente se ejecutará:

```
sudo sysctl -w net.ipv4.ip_forward=1
```

```
ubuntu@ip-10-0-1-160:~$ sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
ubuntu@ip-10-0-1-160:~$ cat /proc/sys/net/ipv4/ip_forward
1
ubuntu@ip-10-0-1-160:~$
```

Para que la configuración se mantenga después de reiniciar el servidor, se editará el archivo `/etc/sysctl.conf`:

```
sudo nano /etc/sysctl.conf
```

En este archivo se modificará el parámetro `net.ipv4.ip_forward` para dejarlo con el siguiente valor:

```
net.ipv4.ip_forward=1
```

```
# Uncomment the next two lines to enable Spoof protection (reverse-path filter)
# Turn on Source Address Verification in all interfaces to
# prevent some spoofing attacks
#net.ipv4.conf.default.rp_filter=1
#net.ipv4.conf.all.rp_filter=1

# Uncomment the next line to enable TCP/IP SYN cookies
# See http://lwn.net/Articles/277146/
# Note: This may impact IPv6 TCP sessions too
#net.ipv4.tcp_syncookies=1

# Uncomment the next line to enable packet forwarding for IPv4
net.ipv4.ip_forward=1
```

Por último, se aplicará la nueva configuración con el siguiente comando:

```
sudo sysctl -p
```

7.6 Configuración de NAT para WireGuard

Una vez configurada la interfaz de WireGuard, se editará el archivo wg0.conf para añadir las reglas de iptables que se ejecutarán automáticamente al iniciar y detener el servicio.

Estas reglas permitirán reenviar el tráfico entre la interfaz de la VPN (wg0) y la interfaz de red del servidor (ens5). Además, se configurará la regla MASQUERADE, que realizará la traducción de direcciones (NAT) para que los equipos conectados a la VPN puedan acceder a Internet utilizando la dirección IP pública de la instancia.

```
SaveConfig = true

PostUp = iptables -A FORWARD -i wg0 -j ACCEPT
PostUp = iptables -A FORWARD -o wg0 -j ACCEPT
PostUp = iptables -t nat -A POSTROUTING -o ens5 -j MASQUERADE

PostDown = iptables -D FORWARD -i wg0 -j ACCEPT
PostDown = iptables -D FORWARD -o wg0 -j ACCEPT
PostDown = iptables -t nat -D POSTROUTING -o ens5 -j MASQUERADE
```

Después de guardar los cambios, se reiniciará el servicio de WireGuard para aplicar la nueva configuración.

Por último, se comprobará que la interfaz se ha iniciado correctamente y que permanece activa

```
ubuntu@ip-10-0-1-160:~$ sudo systemctl restart wg-quick@wg0
ubuntu@ip-10-0-1-160:~$ sudo wg
interface: wg0
  public key: 73lcEe4aKSnp93BzfJ/sLMampFQy84hF9yT3Shguj4=
  private key: (hidden)
  listening port: 51820
ubuntu@ip-10-0-1-160:~$
```

7.7 Configuración del cliente WireGuard

Una vez configurado el servidor, se preparará el equipo cliente para que pueda conectarse a la VPN. Para ello, primero se generará un par de claves, formado por una clave privada y otra pública, que identificarán de forma segura al cliente.

Como el equipo cliente también utiliza Ubuntu, las claves se generarán en el ordenador local con los siguientes comandos: (Si no está instalado Wireguard se tendrá que instalar)

```
mkdir -p ~/wireguard-client
cd ~/wireguard-client
```

```
mtk@mtk-IdeaPad-3-15ITL6:~/aws$ mkdir -p wireguard-client
mtk@mtk-IdeaPad-3-15ITL6:~/aws$ cd wireguard-client/
```

```
wg genkey | tee client_private.key | wg pubkey > client_public.key
```

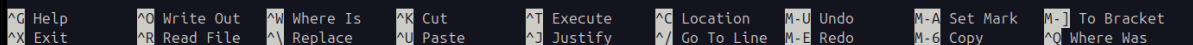
Después, se comprobará que ambos archivos se han creado correctamente:

```
mtk@ntk-IdeaPad-3-15ITL6:~/aws/wireguard-client$ wg genkey | tee client_private.key | wg pubkey > client_public.key
mtk@ntk-IdeaPad-3-15ITL6:~/aws/wireguard-client$ ls -l
total 8
-rw-rw-r-- 1 mtk mtk 45 jul 31 16:10 client_private.key
-rw-rw-r-- 1 mtk mtk 45 jul 31 16:10 client_public.key
mtk@ntk-IdeaPad-3-15ITL6:~/aws/wireguard-client$ sudo chmod 600 client_public.key
mtk@ntk-IdeaPad-3-15ITL6:~/aws/wireguard-client$ sudo chmod 600 client_private.key
mtk@ntk-IdeaPad-3-15ITL6:~/aws/wireguard-client$ ls -l
total 8
-rw----- 1 mtk mtk 45 jul 31 16:10 client_private.key
-rw----- 1 mtk mtk 45 jul 31 16:10 client_public.key
mtk@ntk-IdeaPad-3-15ITL6:~/aws/wireguard-client$
```

Una vez generadas las claves del cliente, será necesario registrarlo en el servidor para que pueda establecer la conexión VPN. Para ello se utilizará la clave pública del cliente, que identificará de forma única el equipo autorizado.

A continuación, en el servidor se editará el archivo de configuración de Wire Guard, al final del archivo se añadirá la configuración del nuevo cliente:

```
[Peer]
PublicKey = kNi1z1k276aZgPdKrGTPBu+nyaiUFMMcKlrHGIM17TE=
AllowedIPs = 10.10.10.2/32
```



La línea PublicKey contendrá la clave pública generada en el equipo cliente, mientras que AllowedIPs asignará al cliente la dirección 10.10.10.2 dentro de la red privada de la VPN.

Una vez guardados los cambios, se reiniciará el servicio para aplicar la nueva configuración. Por último, se comprobará que WireGuard continúa funcionando correctamente .

```
ubuntu@ip-10-0-1-160:~$ sudo systemctl restart wg-quick@wg0
ubuntu@ip-10-0-1-160:~$ sudo wg
interface: wg0
  public key: 73lcEe4aKSnpg93BzfJ/sLMampFQy84hF9yT3Shguj4=
  private key: (hidden)
  listening port: 51820
ubuntu@ip-10-0-1-160:~$
```

Una vez registrado el cliente en el servidor, se creará el archivo de configuración que permitirá establecer la conexión VPN entre el equipo local y la máquina de AWS.

Lo primero será obtener la clave pública del servidor , también será necesario anotar la dirección IPv4 pública de la instancia, ya que se utilizará para que el cliente pueda localizar el servidor a través de Internet.

Después en el equipo local se creará el archivo client.conf.

```
nano ~/aws/wireguard-client/client.conf
```

En este archivo se añadirá la siguiente configuración:

[Interface]

PrivateKey = CLAVE_PRIVADA_DEL_CLIENTE

```
Address = 10.10.10.2/24
DNS = 1.1.1.1

[Peer]
PublicKey = dHChEGUirWHRZl8VYQcc3j050KhRaE8LYzJeXga8nzc=
Endpoint = 52.211.30.202:51820
AllowedIPs = 0.0.0.0/0
PersistentKeepalive = 25
```

Cada uno de estos parámetros tiene una función concreta. La PrivateKey identifica de forma segura al cliente, y Address asigna la dirección IP que utilizará dentro de la VPN. El parámetro DNS indica el servidor DNS que se empleará durante la conexión.

En la sección Peer, la PublicKey permite verificar la identidad del servidor y Endpoint indica la dirección pública y el puerto donde WireGuard está escuchando. AllowedIPs = 0.0.0.0/0 hará que todo el tráfico del equipo pase a través del túnel VPN, y PersistentKeepalive = 25 ayudará a mantener la conexión activa cuando el cliente esté en un router con NAT.

7.8 Conexión del cliente a la VPN

Una vez creado el archivo de configuración del cliente, el siguiente paso será establecer la conexión con el servidor y comprobar que la VPN funciona correctamente.

Primero se copiará el archivo client.conf al directorio de configuración de WireGuard y se renombrará como wg0.conf:

```
sudo cp ~/aws/wireguard-client/client.conf /etc/wireguard/wg0.conf
```

Después, se modificarán los permisos del archivo para que solo pueda acceder a él el usuario administrador, ya que contiene la clave privada del cliente:

```
sudo chmod 600 /etc/wireguard/wg0.conf
```

```
mtk@ntk-IdeaPad-3-15ITL6:~/aws$ sudo cp ~/aws/wireguard-client/client.conf /etc/wireguard/wg0.conf
mtk@ntk-IdeaPad-3-15ITL6:~/aws$ sudo chmod 600 /etc/wireguard/wg0.conf
mtk@ntk-IdeaPad-3-15ITL6:~/aws$
```

Una vez preparado el archivo, se iniciará la VPN con el siguiente comando:

```
sudo wg-quick up wg0
```

```
ntk@ntk-IdeaPad-3-15ITL6:~/aws$ sudo wg-quick up wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.10.10.2/24 dev wg0
[#] ip link set mtu 1420 up dev wg0
[#] resolvconf -a wg0 -n 0 -x
[#] wg set wg0 fwmark 51820
[#] ip -4 rule add not fwmark 51820 table 51820
[#] ip -4 rule add table main suppress_prefixlength 0
[#] ip -4 route add 0.0.0.0/0 dev wg0 table 51820
[#] sysctl -q net.ipv4.conf.all.src_valid_mark=1
[#] nft -f /dev/fd/63
```

Si la configuración es correcta, WireGuard creará la interfaz wg0 y establecerá el túnel VPN con el servidor.

Para comprobar que la conexión se ha establecido correctamente, se ejecutará el siguiente comando tanto en el equipo cliente como en el servidor:

sudo wg

```
ubuntu@ip-10-0-1-160:~$ sudo wg
interface: wg0
  public key: 73lcEe4aKSnpG93BzfJ/sLMampFQy84hF9yT3Shguj4=
  private key: (hidden)
  listening port: 51820

peer: kNI1z1k276aZgPdKrGTpBu+nyaiUFMMcKlRHGIM17TE=
  endpoint: 79.152.205.50:56834
  allowed ips: 10.10.10.2/32
  latest handshake: 1 minute, 56 seconds ago
  transfer: 3.78 MiB received, 1.74 MiB sent
```

En ambos equipos deberá aparecer un latest handshake, lo que indicará que el cliente y el servidor se están comunicando correctamente.

Por último, se realizará una prueba de conectividad enviando un ping a la dirección 10.10.10.1, correspondiente al servidor dentro de la VPN:

ping 10.10.10.1

```
ntk@ntk-IdeaPad-3-15ITL6:~/aws$ ping -c 4 10.10.10.1
PING 10.10.10.1 (10.10.10.1) 56(84) bytes of data.
64 bytes from 10.10.10.1: icmp_seq=1 ttl=64 time=44.9 ms
64 bytes from 10.10.10.1: icmp_seq=2 ttl=64 time=187 ms
64 bytes from 10.10.10.1: icmp_seq=3 ttl=64 time=136 ms
^C
--- 10.10.10.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 44.851/122.726/187.386/58.935 ms
```

Si se recibe respuesta, significará que el túnel VPN se ha establecido correctamente y que la comunicación entre ambos equipos funciona sin problemas, lo podemos comprobar conectando a través de SSH por el túnel.

```

mtk@mtk-IdeaPad-3-15ITL6:~/aws$ ssh -i ~/aws/Aegis-Key.pem ubuntu@10.10.10.1
The authenticity of host '10.10.10.1 (10.10.10.1)' can't be established.
ED25519 key fingerprint is SHA256:5/hahJyzJMcnLLZi4IE9bmcxb2LcVNsBD7/8ckII4N8.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:20: [hashed name]
  ~/.ssh/known_hosts:23: [hashed name]
  ~/.ssh/known_hosts:24: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.10.10.1' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 7.0.0-1009-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Aug  4 09:57:14 UTC 2026

System load:  0.0           Temperature:   -273.1 C
Usage of /:   38.5% of 6.71GB Processes:      120
Memory usage: 29%          Users logged in: 1
Swap usage:   0%           IPv4 address for ens5: 10.0.1.160

 * Ubuntu Pro delivers the most comprehensive open source security and
   compliance features.

   https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Tue Aug  4 09:13:09 2026 from 79.152.205.50
ubuntu@10.0.1.160: ~$

```

7.9. Incidencias encontradas y su resolución

Durante el desarrollo del proyecto surgieron varios problemas que hicieron necesario revisar la configuración de AWS, WireGuard y del propio servidor. Resolver estas incidencias ayudó a entender mejor cómo funcionan todos estos componentes y la importancia de comprobar cada paso antes de continuar con la configuración.

7.9.1 Cambio de la dirección IP pública de la instancia

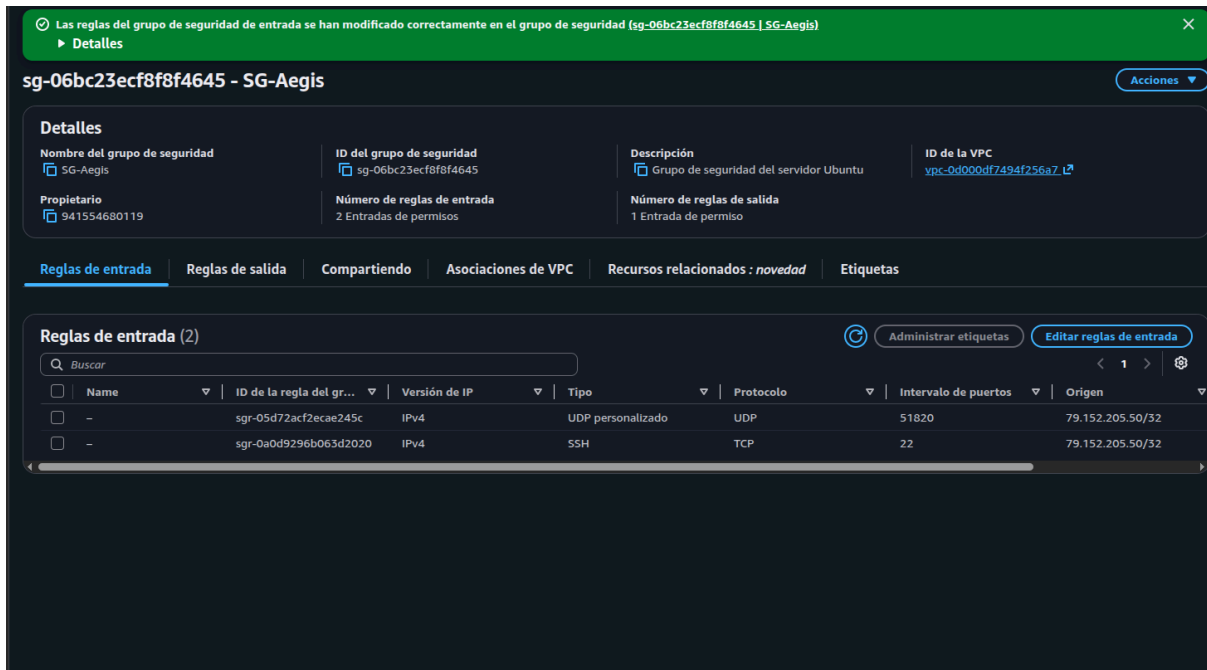
Después de detener y volver a iniciar la instancia EC2, AWS asignó una nueva dirección IP pública. Como el cliente de WireGuard seguía intentando conectarse a la dirección anterior, la VPN dejó de funcionar.

Para solucionarlo se actualizó la dirección IP en el parámetro Endpoint del archivo client.conf y la conexión volvió a funcionar con normalidad.

7.9.2 Puerto UDP 51820 bloqueado en el Security Group

La VPN no llegaba a conectarse y en WireGuard no aparecía el latest handshake.

Se revisó el Security Group de la instancia y se añadió una regla para permitir el tráfico UDP por el puerto 51820, utilizado por WireGuard.



7.9.3 Puerto bloqueado por UFW

Después de abrir el puerto en el Security Group, la VPN seguía sin funcionar.

Se comprobó que el firewall UFW también estaba bloqueando el puerto. Para solucionarlo se añadió la siguiente regla:

`sudo ufw allow 51820/udp`

```
ubuntu@ip-10-0-1-160:~$ sudo ufw status
Status: active

To Action From
--
OpenSSH ALLOW Anywhere
51820/udp ALLOW Anywhere
OpenSSH (v6) ALLOW Anywhere (v6)
51820/udp (v6) ALLOW Anywhere (v6)
```

7.9.4 Lecciones aprendidas

La mayoría de los problemas no estaban relacionados con WireGuard, sino con la configuración de varios componentes que tenían que trabajar juntos. Este proyecto me permitió entender mejor cómo se relacionan AWS, los Security Groups, el firewall UFW, el encaminamiento de red y la VPN, además de la importancia de ir comprobando cada cambio antes de seguir con el siguiente paso.

8. Despliegue de servicios y monitorización

8.1 Instalación de Docker

Una vez que el servidor esté preparado y la VPN funcione correctamente, se instalará Docker para poder ejecutar aplicaciones mediante contenedores.

Docker facilita el despliegue de servicios, ya que cada aplicación funciona de forma independiente y con todas las dependencias que necesita. En este proyecto se utilizará para poner en marcha un servidor web Nginx, que servirá para comprobar que la infraestructura funciona.

Primero actualizamos el sistema e instalamos Docker.

sudo apt install docker.io -y

```
ubuntu@ip-10-0-1-160:~$ sudo apt update && sudo apt upgrade -y
Hit:1 http://eu-west-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://eu-west-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://eu-west-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 http://eu-west-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [180 kB]
Get:6 http://eu-west-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1680 kB]
Get:7 http://eu-west-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [388 kB]
Get:8 http://eu-west-1.ec2.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:9 http://eu-west-1.ec2.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [5772 B]
Get:10 http://eu-west-1.ec2.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Packages [31.0 kB]
Get:11 http://eu-west-1.ec2.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [12.6 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [46.3 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [76.3 kB]
Fetched 2800 kB in 1s (2841 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
ubuntu@ip-10-0-1-160:~$ sudo apt install docker.io -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  bridge-utils containerd dns-root-data dnsmasq-base pigz runc ubuntu-fan
Suggested packages:
```

Cuando termine la instalación, se comprobará que Docker se ha instalado y se verificará que el servicio está en funcionamiento y se configurará para que se inicie automáticamente cada vez que arranque el servidor

```
ubuntu@ip-10-0-1-160:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu3-24.04.2
ubuntu@ip-10-0-1-160:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-08-04 10:24:21 UTC; 2min 11s ago
   TriggeredBy: ● docker.socket
     Docs: https://docs.docker.com
    Main PID: 2723 (dockerd)
      Tasks: 8
     Memory: 59.8M (peak: 60.6M)
        CPU: 468ms
    CGroup: /system.slice/docker.service
            └─2723 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Aug 04 10:24:21 ip-10-0-1-160 dockerd[2723]: time="2026-08-04T10:24:21.337302889Z" level=info msg="Restoring containers: start."
Aug 04 10:24:21 ip-10-0-1-160 dockerd[2723]: time="2026-08-04T10:24:21.367779049Z" level=info msg="Deleting nftables IPv4 rules" error="e
Aug 04 10:24:21 ip-10-0-1-160 dockerd[2723]: time="2026-08-04T10:24:21.376994701Z" level=info msg="Deleting nftables IPv6 rules" error="e
Aug 04 10:24:21 ip-10-0-1-160 dockerd[2723]: time="2026-08-04T10:24:21.677274642Z" level=info msg="Loading containers: done."
Aug 04 10:24:21 ip-10-0-1-160 dockerd[2723]: time="2026-08-04T10:24:21.702577611Z" level=info msg="Docker daemon" commit="29.1.3-0ubuntu3
Aug 04 10:24:21 ip-10-0-1-160 dockerd[2723]: time="2026-08-04T10:24:21.703437131Z" level=info msg="Initializing buildkit"
Aug 04 10:24:21 ip-10-0-1-160 dockerd[2723]: time="2026-08-04T10:24:21.703437131Z" level=info msg="Completed buildkit initialization"
```

Se ejecutará el contenedor hello-world, es la imagen oficial de prueba de Docker para confirmar que todo funciona.

```
ubuntu@ip-10-0-1-160:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:c3cbe1cc1aa588a64951ac6286e0df7b27fe2e6324b1001c619bb358770c0178
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
```

8.2 Despliegue de un servidor web con Nginx mediante Docker

Una vez comprobado que Docker funciona bien, se desplegará el primer servicio del proyecto utilizando un contenedor.

Para ello se utilizará Nginx, uno de los servidores web más utilizados tanto en entornos profesionales como en la nube. En lugar de instalarlo directamente sobre Ubuntu, se ejecutará dentro de un contenedor Docker. Así, el servicio quedará separado del sistema operativo, lo que facilitará su administración y permitirá eliminarlo o volver a desplegarlo de forma rápida si fuera necesario.

Con este despliegue también se comprobará que la infraestructura creada en AWS está preparada para alojar aplicaciones en contenedores y que el servidor puede publicar servicios accesibles desde la red.

8.2.1 Creación del contenedor

Para desplegar el servidor web, se ejecutará el siguiente comando:

```
sudo docker run -d --name nginx -p 80:80 --restart unless-stopped nginx:latest
```

```
ubuntu@ip-10-0-1-160:~$ sudo docker run -d --name nginx -p 80:80 --restart unless-stopped nginx:latest
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
062e450697fa: Pull complete
b6698f04e005: Pull complete
d26f27cc8c41: Pull complete
3c7ab7949321: Pull complete
2bedaf25031a: Pull complete
32454cdbf456: Pull complete
cacfcdd01f30: Pull complete
5c496f5b5050: Download complete
aa1d76ccc2c6: Download complete
Digest: sha256:5a88c9c45479443d7be2eadc894b4ed0a9801bae03d97a5760ae13b5c2005942
Status: Downloaded newer image for nginx:latest
1a2d87da83c5074b3b937ee8c7c32873a819703fce30b30e6af2e8fa584036fd
ubuntu@ip-10-0-1-160:~$
```

Con este comando, el contenedor se ejecutará en segundo plano, recibirá el nombre nginx y publicará el puerto 80, lo que permitirá acceder al servidor web desde el exterior.

Además, quedará configurado para iniciarse automáticamente cada vez que el servidor se reinicie

Si la imagen de Nginx no está disponible en el servidor, Docker la descargará automáticamente desde Docker Hub antes de crear el contenedor.

Una vez creado el contenedor, se comprobará que está funcionando.

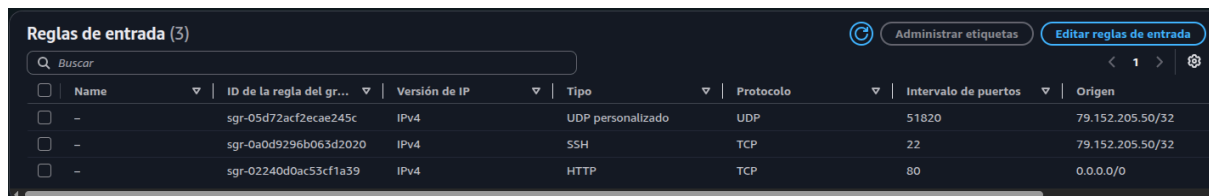
`sudo docker ps`

```
ubuntu@ip-10-0-1-160:~$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
1a2d87da83c5   nginx:latest   "/docker-entrypoint..." 3 minutes ago  Up 3 minutes  0.0.0.0:80->80/tcp, [::]:80->80/tcp  nginx
```

8.2.2 Publicación del servicio web

Una vez creado el contenedor, el siguiente paso será permitir el acceso al servidor web desde Internet.

Para ello, se añadirá una regla en el Security Group de AWS para permitir las conexiones por el puerto 80, utilizado por el servicio HTTP.



The screenshot shows the AWS Management Console 'Reglas de entrada' (Inbound Rules) page for a security group. It contains three rules. The third rule is highlighted, showing it allows HTTP traffic on port 80 from any source (0.0.0.0/0).

	Name	ID de la regla del gr...	Versión de IP	Tipo	Protocolo	Intervalo de puertos	Origen
☐	-	sgr-05d72acf2ecae245c	IPv4	UDP personalizado	UDP	51820	79.152.205.50/32
☐	-	sgr-0a0d9296b063d2020	IPv4	SSH	TCP	22	79.152.205.50/32
☐	-	sgr-02240d0ac53cf1a59	IPv4	HTTP	TCP	80	0.0.0.0/0

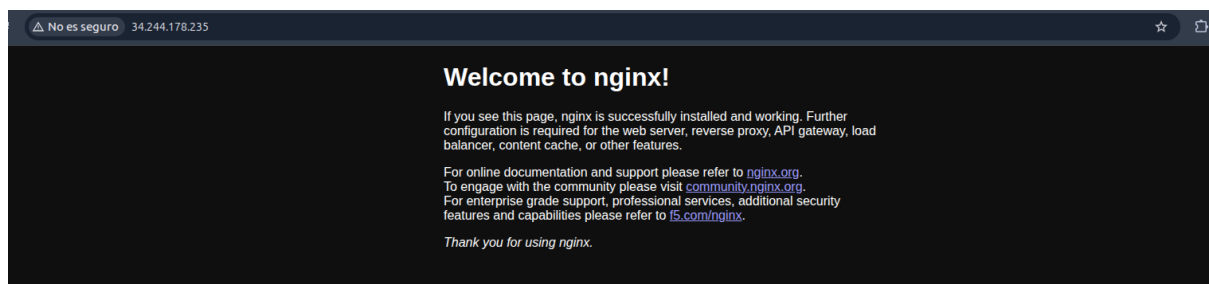
Además, también necesitaremos abrir ese mismo puerto en el firewall UFW del servidor.

`sudo ufw allow 80/tcp`

```
ubuntu@ip-10-0-1-160:~$ sudo ufw allow 80/tcp
Rule added
Rule added (v6)
ubuntu@ip-10-0-1-160:~$ sudo ufw status
Status: active

To Action From
--
OpenSSH ALLOW Anywhere
51820/udp ALLOW Anywhere
80/tcp ALLOW Anywhere
OpenSSH (v6) ALLOW Anywhere (v6)
51820/udp (v6) ALLOW Anywhere (v6)
80/tcp (v6) ALLOW Anywhere (v6)
```

Después de aplicar esta configuración, se podrá acceder a la dirección IP pública de la instancia desde un navegador para comprobar que el servidor web responde, si todo está bien, se mostrará la página de bienvenida de Nginx.



8.2.3 Personalización de la página web

Una vez comprobado que Nginx funciona, se sustituirá la página de bienvenida por una página web creada para este proyecto. Con ello se podrá comprobar que el servidor está mostrando contenido propio y no únicamente la página que incluye Nginx por defecto.

La nueva página tendrá un diseño sencillo e incluirá información sobre la infraestructura desplegada en AWS y las principales tecnologías utilizadas durante el proyecto.

Para realizar el cambio, primero se accederá al interior del contenedor

```
sudo docker exec -it nginx sh
```

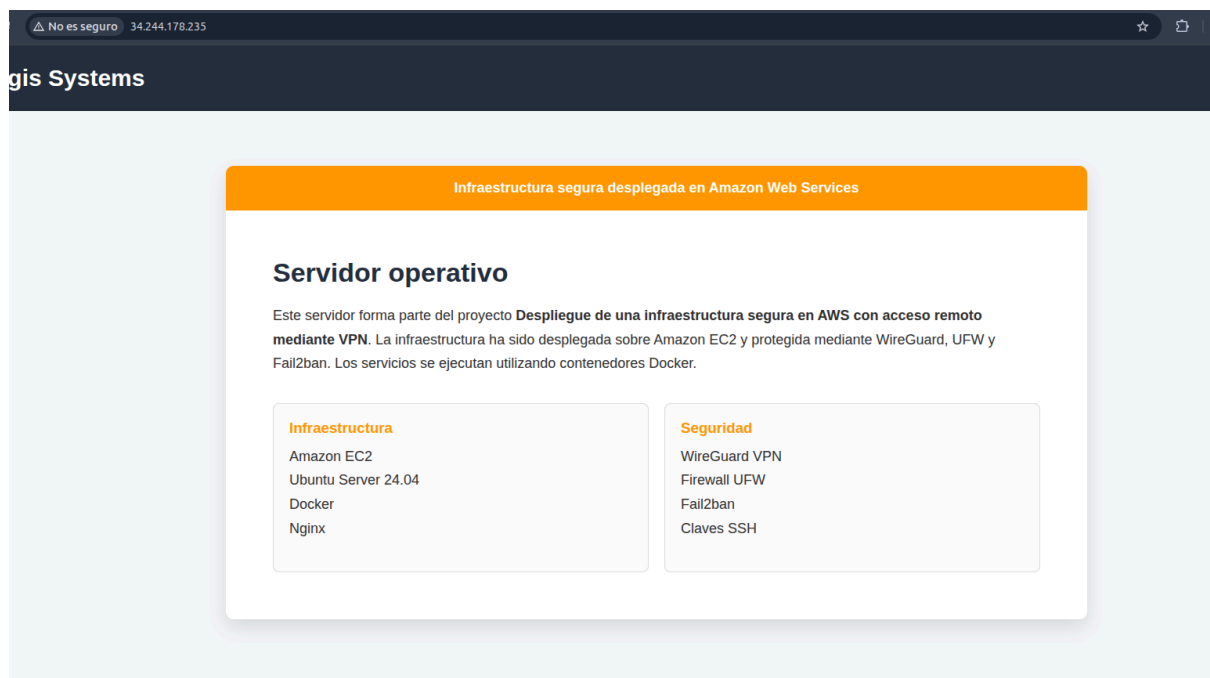
Después, se accederá al directorio donde Nginx almacena los archivos de la página web:

```
cd /usr/share/nginx/html
```

En este directorio se sustituirá el archivo index.html por la página creada para el proyecto.

```
ubuntu@ip-10-0-1-160:~$ sudo docker exec -it nginx sh
# cd /usr/share/nginx/html
# cat > index.html
```

Una vez guardados los cambios, no será necesario reiniciar el contenedor, ya que Nginx mostrará el nuevo contenido de forma inmediata, para comprobarlo, se accederá desde un navegador a la dirección IP pública de la instancia EC2



8.3 Monitorización con Amazon CloudWatch

Una vez desplegados los principales servicios, el siguiente paso será configurar la monitorización del servidor mediante Amazon CloudWatch.

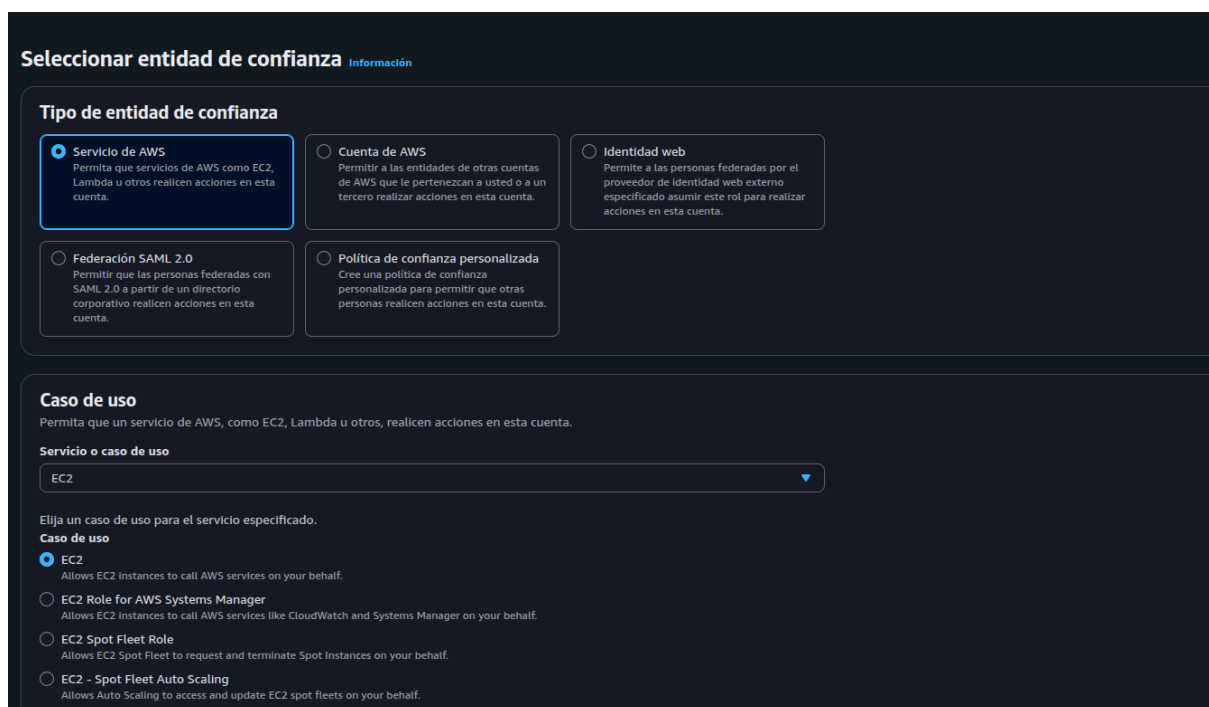
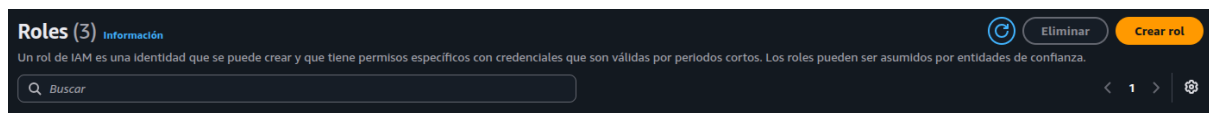
CloudWatch permitirá consultar el estado de la instancia, visualizar métricas como el uso de CPU o la actividad de la red y disponer de información para comprobar el funcionamiento del servidor.

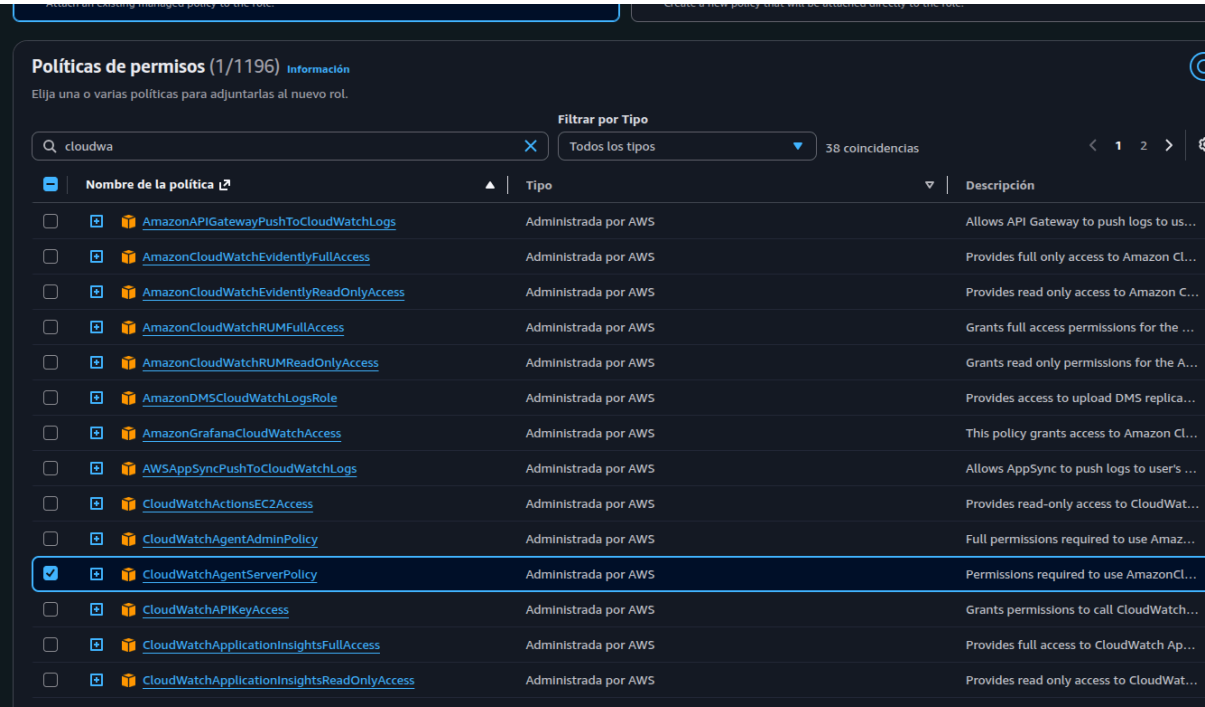
Antes de poder utilizar este servicio, será necesario asignar a la instancia EC2 un rol de IAM con los permisos adecuados. De esta forma, el servidor podrá enviar información a CloudWatch de forma segura, sin necesidad de guardar credenciales dentro del sistema operativo.

8.3.1 Creación del rol IAM

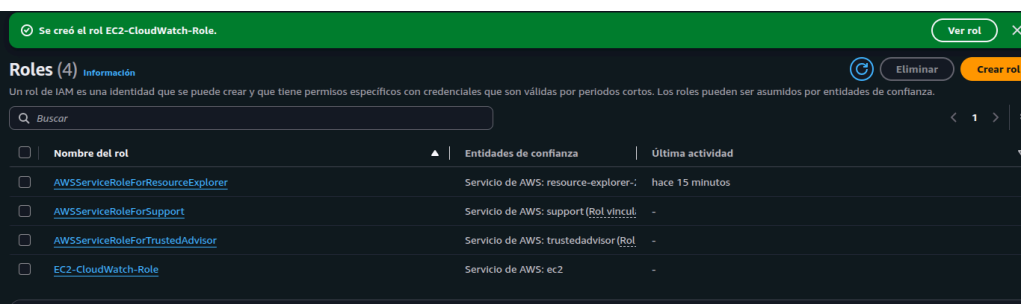
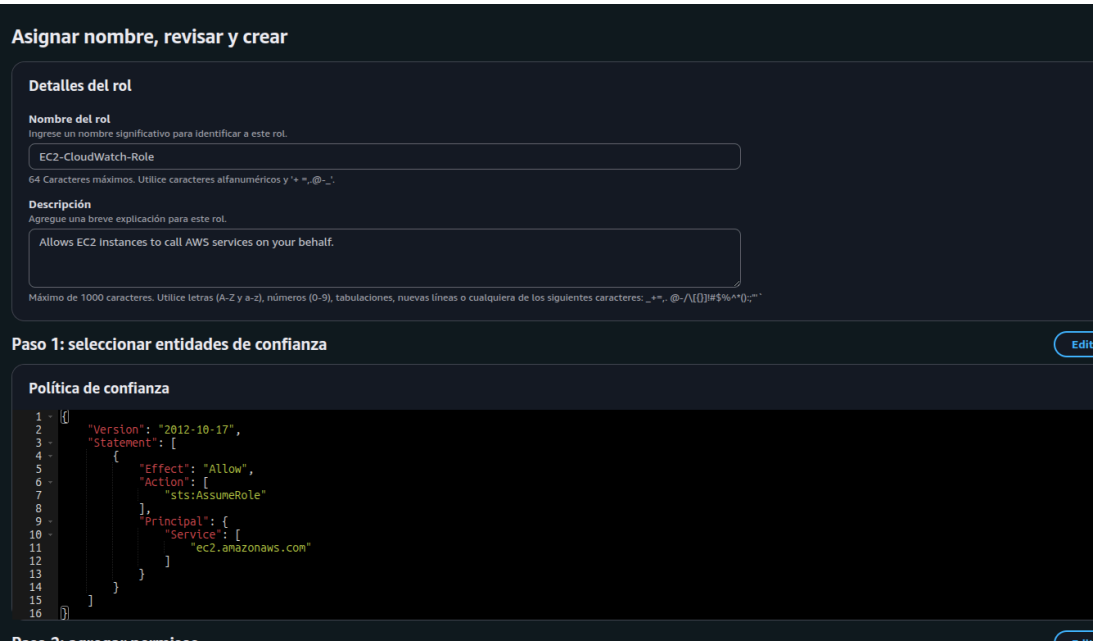
El primer paso será crear un rol de IAM que permita a la instancia EC2 comunicarse con Amazon CloudWatch.

Para ello, desde el servicio IAM se seleccionará la opción Crear rol y se elegirá EC2 como servicio de confianza. Después, se añadirá la política CloudWatchAgentServerPolicy, que proporciona los permisos necesarios para que el servidor pueda enviar métricas y registros a CloudWatch.



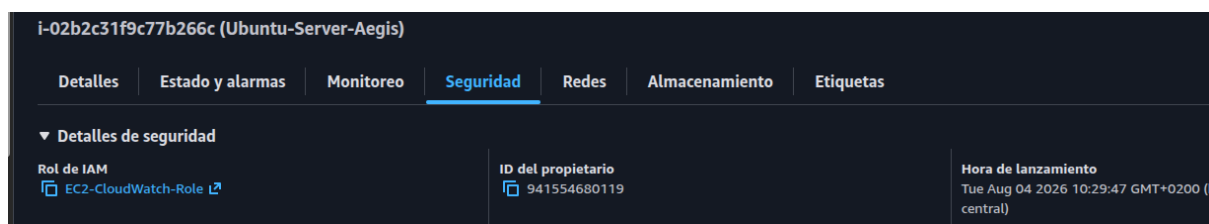
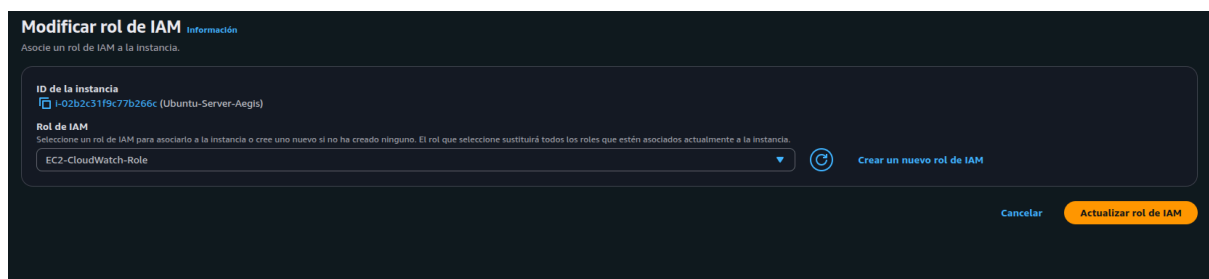
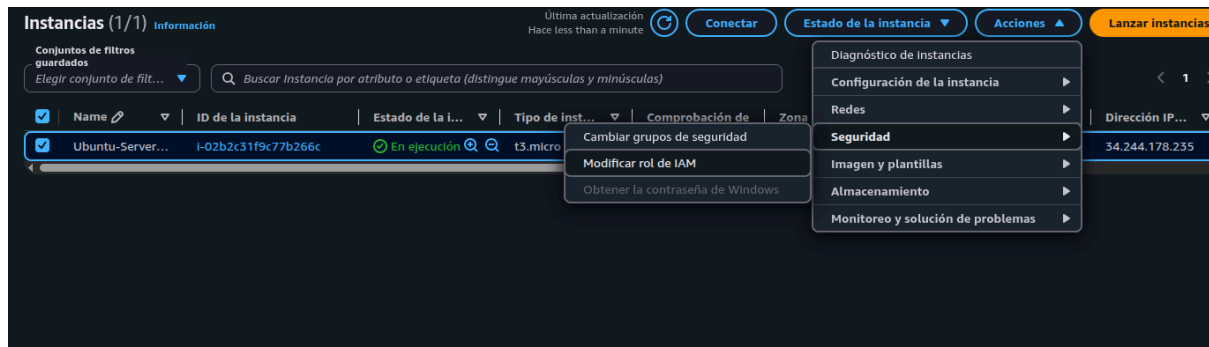


Una vez completada la configuración, el rol se guardará con el nombre EC2-CloudWatch-Role y quedará listo para asociarlo a la instancia EC2.



8.4.2 Asociación del rol a la instancia EC2

Una vez creado el rol de IAM, se asociará a la instancia EC2 para que el servidor pueda utilizar los permisos necesarios sin tener que guardar credenciales de acceso. Así, la instancia podrá comunicarse con Amazon CloudWatch utilizando las credenciales temporales que AWS proporciona automáticamente. De esta forma, el agente de CloudWatch podrá enviar métricas y otra información del servidor de manera segura.



8.4.3 Instalación del agente de CloudWatch

Una vez asociado el rol de IAM a la instancia EC2, se instalará el agente de Amazon CloudWatch, este agente será el encargado de recopilar información del servidor, como el uso de recursos o el estado del sistema, y enviarla a CloudWatch para poder consultarla desde la consola de AWS. De esta forma, será posible supervisar el funcionamiento de la instancia sin necesidad de acceder directamente al servidor.

Al intentar instalar el agente de Amazon CloudWatch desde los repositorios de Ubuntu, no fue posible encontrar el paquete.

```
ubuntu@ip-10-0-1-160:~$ sudo apt install amazon-cloudwatch-agent -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
E: Unable to locate package amazon-cloudwatch-agent
```

Esto se debe a que el agente no está disponible en los repositorios estándar de Ubuntu 24.04, por lo que fue necesario utilizar el paquete oficial proporcionado por Amazon, descargándolo e instalándolo.

wget

<https://amazoncloudwatch-agent.s3.amazonaws.com/ubuntu/amd64/latest/amazon-cloudwatch-agent.deb>

```
ubuntu@ip-10-0-1-160:~$ wget https://amazoncloudwatch-agent.s3.amazonaws.com/ubuntu/amd64/latest/amazon-cloudwatch-agent.deb
--2026-08-04 12:54:11-- https://amazoncloudwatch-agent.s3.amazonaws.com/ubuntu/amd64/latest/amazon-cloudwatch-agent.deb
Resolving amazoncloudwatch-agent.s3.amazonaws.com (amazoncloudwatch-agent.s3.amazonaws.com)... 54.231.199.209, 16.15.207.147, 16.15.244.152, ..
Connecting to amazoncloudwatch-agent.s3.amazonaws.com (amazoncloudwatch-agent.s3.amazonaws.com)|54.231.199.209|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 68733108 (66M) [application/octet-stream]
Saving to: 'amazon-cloudwatch-agent.deb'

amazon-cloudwatch-agent.deb      100%[=====] 65.55M  26.0MB/s   in 2.5s

2026-08-04 12:54:14 (26.0 MB/s) - 'amazon-cloudwatch-agent.deb' saved [68733108/68733108]

ubuntu@ip-10-0-1-160:~$ sudo dpkg -i amazon-cloudwatch-agent.deb
Selecting previously unselected package amazon-cloudwatch-agent.
(Reading database ... 112419 files and directories currently installed.)
Preparing to unpack amazon-cloudwatch-agent.deb ...
create group cwagent, result: 0
create user cwagent, result: 0
```

Cuando se acabó la instalación, se comprobó que el agente estaba disponible en el servidor ejecutando su comando de ayuda.

```
ubuntu@ip-10-0-1-160:~$ amazon-cloudwatch-agent-ctl -h

usage: amazon-cloudwatch-agent-ctl -a
stop|start|status|status-with-workloads|fetch-config|append-config|remove-config|set-log-level
[-m ec2|onPremise|onPrem|auto]
[-c default|all|ssm:<parameter-store-name>|file:<file-path>]
[-s]
[-d]
[-l INFO|DEBUG|WARN|ERROR|OFF]
```

8.4.4 Configuración del agente

Una vez completado el asistente de configuración, el agente generará automáticamente el archivo config.json, donde quedará guardada toda la configuración utilizada por Amazon CloudWatch.

En este archivo se definirá la recopilación de métricas del sistema, como el uso de memoria, el espacio en disco y la actividad de entrada y salida, con una frecuencia de 60 segundos. También se configurará el envío del archivo de registro /var/log/auth.log a CloudWatch Logs, estableciendo una retención de 30 días.

```

ubuntu@ip-10-0-1-160:~$ sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-config-wizard
Starting config-wizard, this will map back to a call to amazon-cloudwatch-agent
Executing /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent with arguments: [config-wizard]=====
=====
= Welcome to the Amazon CloudWatch Agent Configuration Manager =
=
= CloudWatch Agent allows you to collect metrics and logs from =
= your host and send them to CloudWatch. Additional CloudWatch =
= charges may apply. =
=====
On which OS are you planning to use the agent?
1. linux
2. windows
3. darwin
default choice: [1]:

Trying to fetch the default region based on ec2 metadata...
!! imds retry client will retry 1 timesAre you using EC2 or On-Premises hosts?
1. EC2
2. On-Premises
default choice: [1]:

Which user are you planning to run the agent?
1. cwagent
2. root
3. others
default choice: [1]:

Do you want to turn on StatsD daemon?
1. yes
2. no
default choice: [1]:
2

```

El archivo de configuración quedará almacenado en la siguiente ruta:

/opt/aws/amazon-cloudwatch-agent/bin/config.json

```

Current config as follows:
{
  "agent": {
    "metrics_collection_interval": 60,
    "run_as_user": "cwagent"
  },
  "metrics": {
    "append_dimensions": {
      "AutoScalingGroupName": "${aws:AutoScalingGroupName}",
      "ImageId": "${aws:ImageId}",
      "InstanceId": "${aws:InstanceId}",
      "InstanceType": "${aws:InstanceType}"
    },
    "metrics_collected": {
      "disk": {
        "measurement": [
          "used_percent",
          "inodes_free"
        ],
        "metrics_collection_interval": 60,
        "resources": [
          "*"
        ]
      },
      "diskio": {
        "measurement": [
          "io_time"
        ],
        "metrics_collection_interval": 60,
        "resources": [
          "*"
        ]
      },
      "mem": {
        "measurement": [
          "mem_used_percent"
        ]
      }
    }
  }
}

```

Las opciones relacionadas con StatsD, CollectD, AWS X-Ray, la importación de configuraciones anteriores y el almacenamiento de la configuración en AWS Systems Manager Parameter Store se dejaron deshabilitadas, ya que no eran necesarias para los objetivos de este proyecto.

8.4.5 Puesta en marcha del agente

Una vez generado el archivo de configuración, el siguiente paso consistirá en aplicarlo al agente e iniciar el servicio para que comience a enviar métricas y registros a Amazon CloudWatch.

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl  
-a fetch-config -m ec2 -c file:/opt/aws/amazon-cloudwatch-agent/bin/config.json -s
```

```
ubuntu@ip-10-0-1-160:~$ sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a fetch-config -m ec2 -c file:/opt/aws/amazon-cloudwatch-agent/bin/config.json -s
***** processing amazon-cloudwatch-agent *****
Starting config-downloader, this will map back to a call to amazon-cloudwatch-agent
Executing /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent with arguments: [config-downloader -mode ec2 -download-source file:/opt/aws/amazon-cloudwatch-agent/bin/config.json -output-dir /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.d -config /opt/aws/amazon-cloudwatch-agent/etc/common-config.toml -multi-config default]!! Trying to detect region from ec2
D! [EC2] Found active network interface
I! inds retry client will retry 1 times
Start configuration validation...
2026/08/07 09:33:52 Reading json config file path: /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.d/file_config.json.tmp ...
2026/08/07 09:33:52 I! Valid json input schema.
2026/08/07 09:33:52 D! ec2tagger processor required because append_dimensions is set
2026/08/07 09:33:52 D! delta processor required because metrics with diskio or net are set
2026/08/07 09:33:52 D! ec2tagger processor required because append_dimensions is set
2026/08/07 09:33:52 Configuration validation first phase succeeded
Starting config-translator, this will map back to a call to amazon-cloudwatch-agent
Executing /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent with arguments: [config-translator -config /opt/aws/amazon-cloudwatch-agent/etc/common-config.toml -multi-config default -input /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json -input-dir /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.d -output /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.toml -mode ec2]!! Trying to detect region from ec2
D! [EC2] Found active network interface
I! inds retry client will retry 1 times
/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent -schematest -config /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.toml
Configuration validation second phase succeeded
Configuration validation succeeded
amazon-cloudwatch-agent has already been stopped
Created symlink /etc/systemd/system/multi-user.target.wants/amazon-cloudwatch-agent.service → /etc/systemd/system/amazon-cloudwatch-agent.service.
ubuntu@ip-10-0-1-160:~$
```

Este comando cargará el archivo de configuración creado previamente (config.json), lo aplicará al agente de CloudWatch e iniciará automáticamente el servicio.

Explicación del comando

fetch-config: carga la configuración definida en el archivo config.json.

-m ec2: indica que el agente se está ejecutando sobre una instancia Amazon EC2.

-c file:: especifica que la configuración se leerá desde un archivo local.

-s: inicia automáticamente el agente una vez aplicada la configuración.

Una vez ejecutado este comando, el agente comenzará a recopilar las métricas y los registros configurados, enviándolos de forma automática a Amazon CloudWatch.

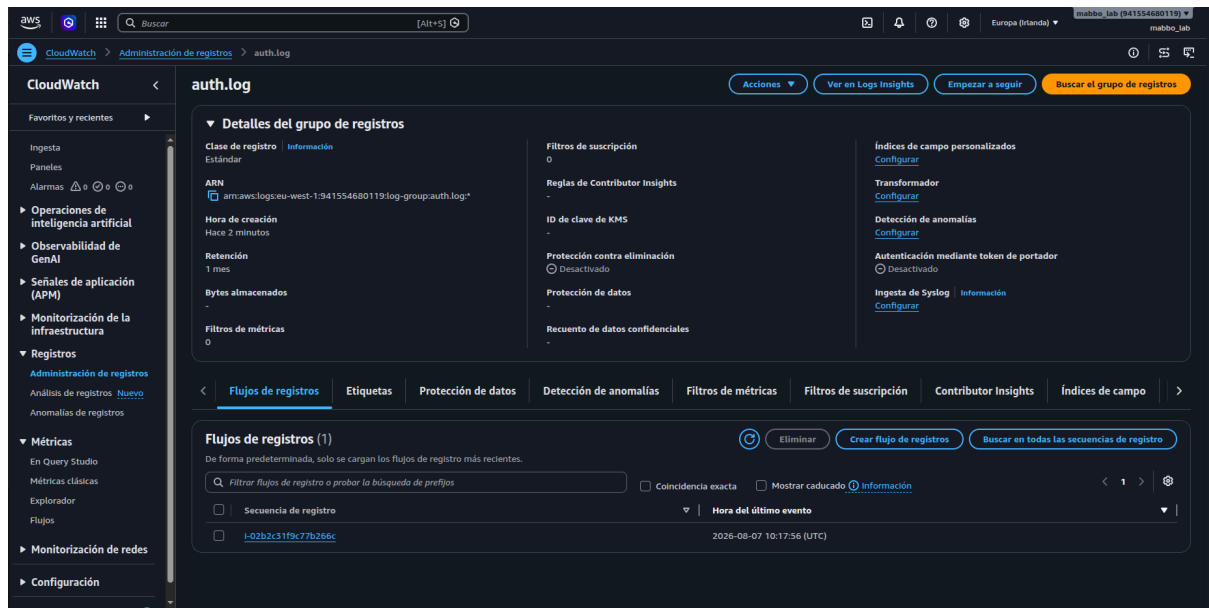
8.4.6 Verificación de la monitorización

Una vez iniciado el agente, se comprobará su estado para confirmar que el servicio está en funcionamiento.

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a status
```

Si todo ha ido bien, el resultado mostrará el estado running, indicando que el agente está activo y que ya puede enviar la información a Amazon CloudWatch, recopilando las métricas y los registros definidos en el archivo de configuración, que podrán consultarse desde la consola de AWS.

Después, se accederá al apartado CloudWatch Logs para comprobar que también se están enviando los registros del servidor. Allí aparecerá el grupo de registros auth.log, donde se almacenarán los eventos de autenticación configurados durante la instalación del agente.

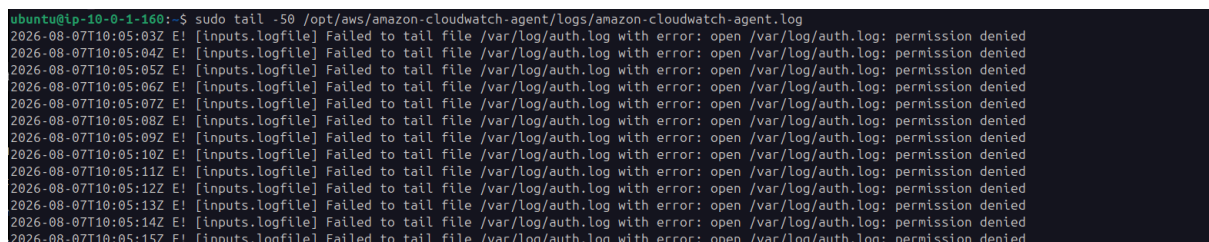


Incidencia durante la configuración de CloudWatch

Durante la configuración de Amazon CloudWatch, las métricas comenzaron a enviarse sin problemas, pero el grupo de registros auth.log no aparecía en CloudWatch Logs.

Después de revisar los registros del agente, se encontró el siguiente mensaje:

Failed to tail file /var/log/auth.log: permission denied



El problema era con los permisos del archivo /var/log/auth.log. El agente de CloudWatch se ejecutaba con el usuario cwagent, que no tenía permisos para leer este archivo.

Para solucionarlo, se añadió el usuario cwagent al grupo adm con el siguiente comando:

sudo usermod -aG adm cwagent

Después de reiniciar el agente, el grupo de registros auth.log apareció en CloudWatch Logs y los eventos comenzaron a enviarse a AWS.

```

ubuntu@ip-10-0-1-160:~$ ls -l /var/log/auth.log
-rw-r----- 1 syslog adm 8729 Aug  7 10:15 /var/log/auth.log
ubuntu@ip-10-0-1-160:~$ sudo usermod -aG adm cwagent
ubuntu@ip-10-0-1-160:~$ groups cwagent
cwagent : cwagent adm
ubuntu@ip-10-0-1-160:~$ sudo systemctl restart amazon-cloudwatch-agent
ubuntu@ip-10-0-1-160:~$ sudo -u cwagent head /var/log/auth.log
2026-08-07T08:24:12.945688+00:00 ip-10-0-1-160 systemd-logind[606]: New seat seat0.
2026-08-07T08:24:12.945697+00:00 ip-10-0-1-160 systemd-logind[606]: Matching system buttons on /dev/input/event0 (Power Button)
2026-08-07T08:24:12.945701+00:00 ip-10-0-1-160 systemd-logind[606]: Matching system buttons on /dev/input/event1 (Sleep Button)
2026-08-07T08:24:12.945706+00:00 ip-10-0-1-160 systemd-logind[606]: Matching system buttons on /dev/input/event2 (AT Translated Set 2 keyboard)
2026-08-07T08:24:12.945728+00:00 ip-10-0-1-160 polkitd[600]: Loading rules from directory /etc/polkit-1/rules.d
2026-08-07T08:24:12.945733+00:00 ip-10-0-1-160 polkitd[600]: Loading rules from directory /usr/share/polkit-1/rules.d
2026-08-07T08:24:12.945738+00:00 ip-10-0-1-160 polkitd[600]: Finished loading, compiling and executing 6 rules
2026-08-07T08:24:12.945753+00:00 ip-10-0-1-160 polkitd[600]: Acquired the name org.freedesktop.PolicyKit1 on the system bus
2026-08-07T08:25:03.018988+00:00 ip-10-0-1-160 CRON[1415]: pam_unix(cron:session): session opened for user root(uid=0) by root(uid=0)
2026-08-07T08:25:03.023752+00:00 ip-10-0-1-160 CRON[1415]: pam_unix(cron:session): session closed for user root
ubuntu@ip-10-0-1-160:~$

```

9. Conclusiones

Con este proyecto he podido diseñar y desplegar una infraestructura segura en Amazon Web Services (AWS). A lo largo del trabajo se ha creado una red en la nube, se ha configurado un servidor Ubuntu y se han ido incorporando diferentes medidas de seguridad hasta conseguir una infraestructura que funciona.

Durante el desarrollo he trabajado con servicios como Amazon VPC, EC2, IAM y CloudWatch, además de herramientas como WireGuard, Docker, UFW y Fail2ban. Esto me ha permitido conocer mejor cómo se integran estos componentes y cómo se puede desplegar un servidor de forma segura en un entorno cloud.

Sin duda, la parte de la que más he aprendido ha sido la resolución de los problemas que fueron apareciendo durante el proyecto. En varias ocasiones tuve que revisar configuraciones, buscar el origen de algunos errores y probar distintas soluciones hasta conseguir que todo funcionara. Aunque al principio supuso un reto, este proceso me ayudó a entender mucho mejor el funcionamiento de AWS y a ganar confianza a la hora de resolver incidencias.

También he podido reforzar conocimientos sobre administración de sistemas Linux, redes y ciberseguridad, además de familiarizarme con herramientas que se utilizan habitualmente en el ámbito profesional. Todo ello ha hecho que el proyecto no solo sirviera para alcanzar los objetivos planteados, sino también para seguir aprendiendo y adquirir experiencia práctica.

En conjunto, considero que el resultado ha sido muy positivo. Además de conseguir una infraestructura segura y completamente funcional, este proyecto me ha permitido aplicar conocimientos de diferentes áreas en un mismo entorno y entender cómo trabajan juntas las distintas tecnologías utilizadas.